

一、互联网概述

1.1 当前整个互联网发展现状

- 1 整个社会大背景其实处于经济复苏的阶段，互联网现状由于其他行业的影响，导致，也出人员‘过剩’，所谓的人员过剩只不过是低级或者初级开发人员，过剩。由于人工智能AI发展的，很多繁琐的工作都可以使用人工智能进行完成，行业大批量裁员，主要针对于初级程序员，就会导致行业的内卷严重。而且任何一个行业的发展，都会经历波峰波谷的阶段，咱们只需要在这个阶段中不断地去充实自己，就可以在将来或者未来行业发展快速时候，进入快车道，实现弯道超车。
- 2 当下大学生如何应对负责就业环境，任何一个行业对于人才需求都是巨大的，只需要不断地提升自己，提升自我的能力和技术水准，任然可以有自己的一席之地。
- 3 当前互联网行业，应该如何选择就业方向。目前来说整个互联网行业，以开发岗位作为介绍：
- 4 云计算：云计算脱胎与运维或者实施
- 5 大数据：从2016年开始，‘大数据’，在2018年，大数据方向进入寒冬期，大数据行业存在就业难得情况
- 6 人工智能：人工智能一定是将来大热、大模型的发展
- 7 python数据分析：python数据分析，python分析的比重也会越来越大
- 8 物联网：智能家居，小米su7，万物物联
- 9 A-R:虚拟现实
- 10 嵌入式：整体比较难
- 11 go语言开发：近两年比较新的行业

1.2 python现状

- 1 python语言面向对象语言，应用领域非常广。
- 2 数据爬虫：很多企业中都是必备岗位
- 3 数据分析：对于数据进行数据清洗、数据分析、数据挖掘
- 4 人工智能：机器和程序像人一样。思考、学习、看、读、说等，声音识别、图像识别、深度学习（冒险岛）
- 5 web框架：python本身提供flask框架、django框架
- 6 python语言来说，在整个当前互联网，是排行榜首

1.3开发工具

- 1 python的库3.9
- 2 python的开发工具pycharm

	psiphon3.exe 715.8 MB 0次 剩余 10 天过期	11:28 来自: error→_→bug
	Crack.zip 95.4 KB 0次	11:29 来自: error→_→bug
	python-3.9.5-amd64.exe 27.1 MB 0次	11:28 来自: error→_→bug

二、开发环境搭建

2.1 python编译环境安装

- ¹ 访问对应技术的官网，在对应的官网可以进行相关的下载工作：<https://www.python.org/>
- ² 编译包下载后，在本地进行安装

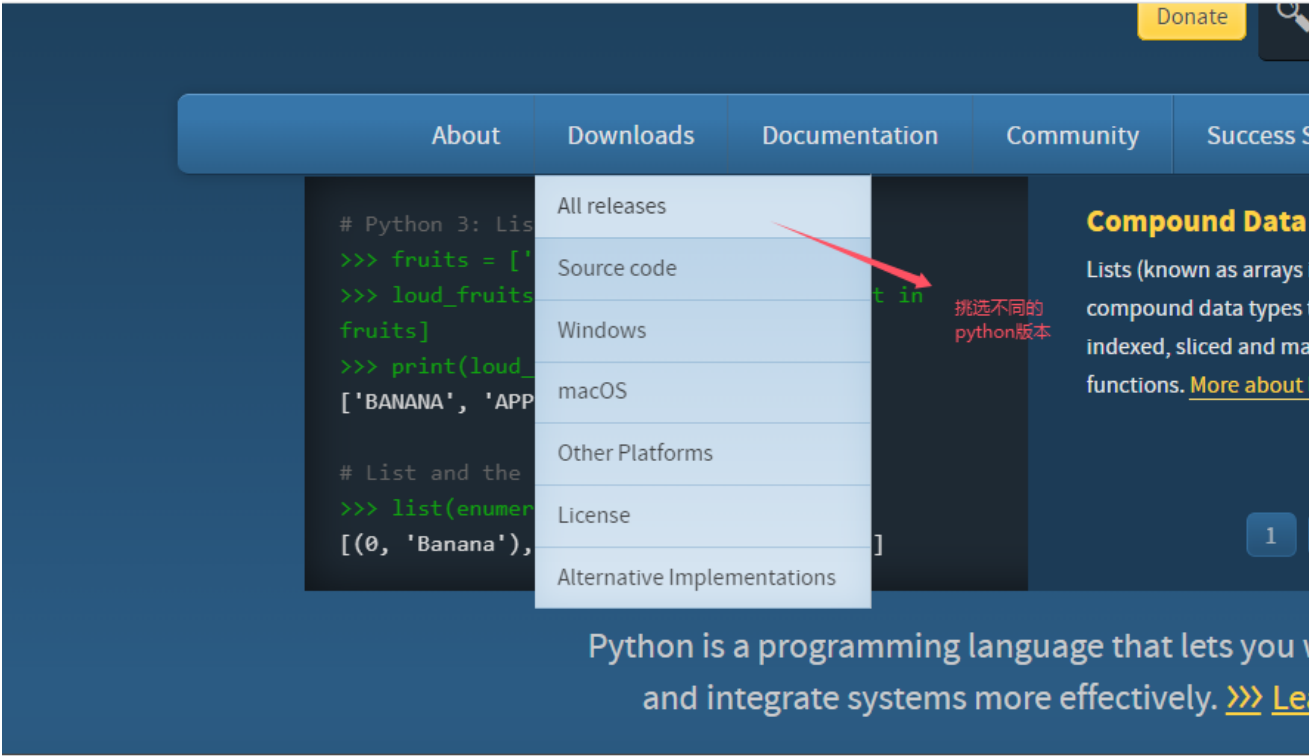
最新版：



Download

Python source code and installers are available for download for all versions!

Latest: Python 3.13.1



3.8	end of life, last release was 3.8.20	2019-10-14	2024-10-07	PEP 569
-----	--------------------------------------	------------	------------	---------

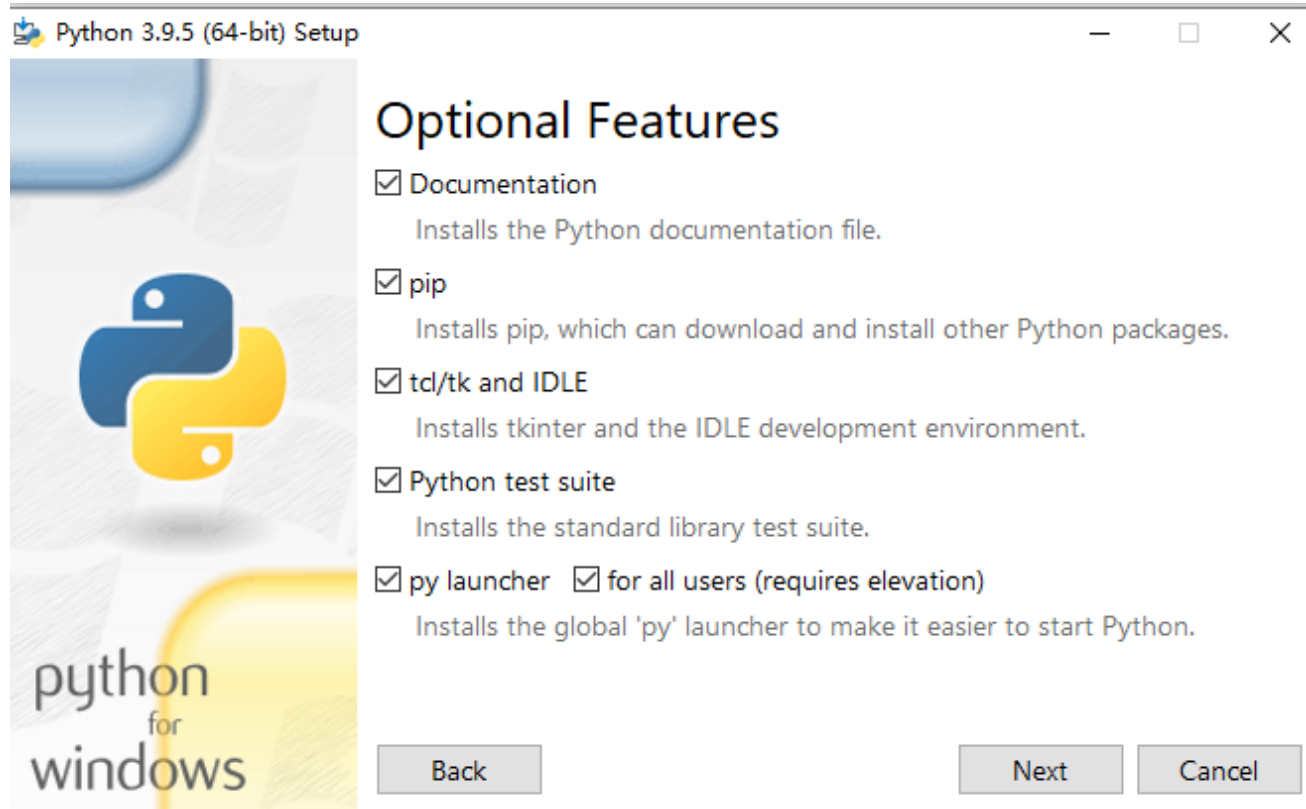
Looking for a specific release?

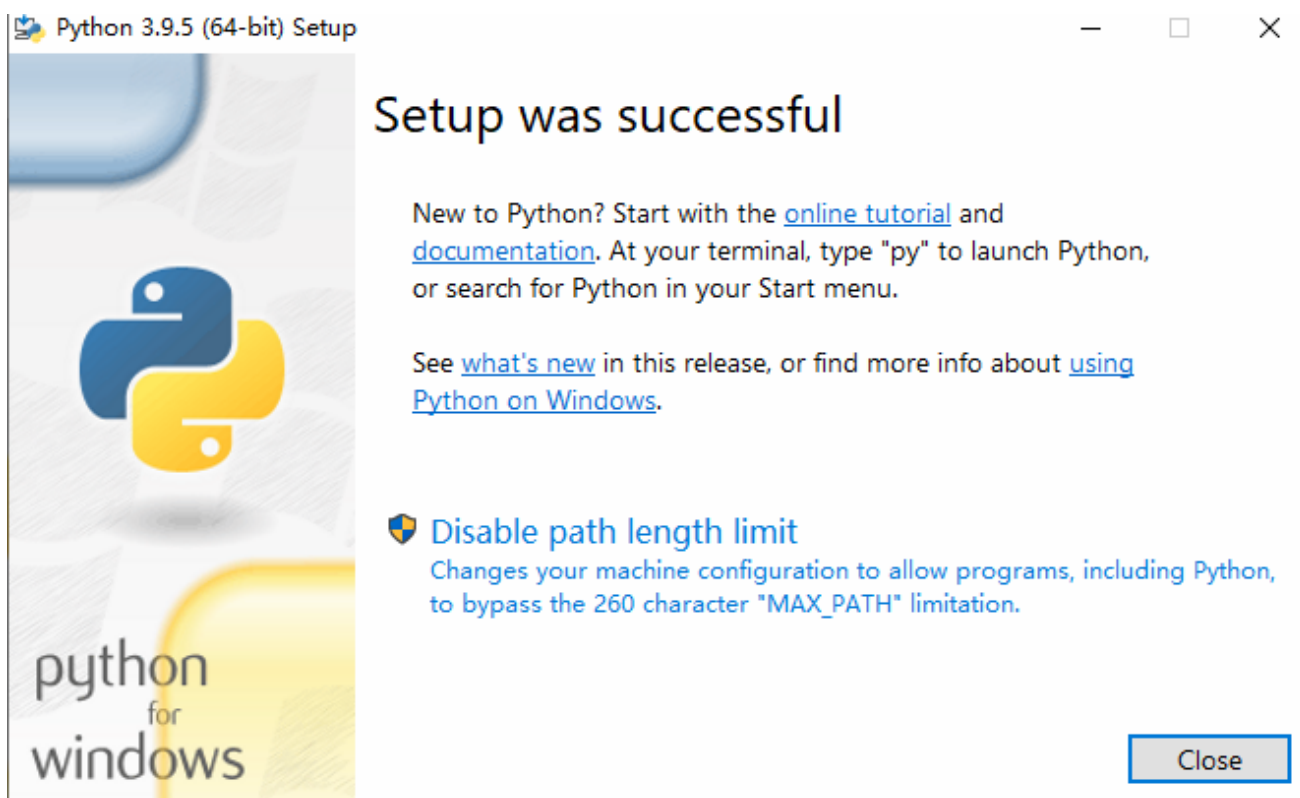
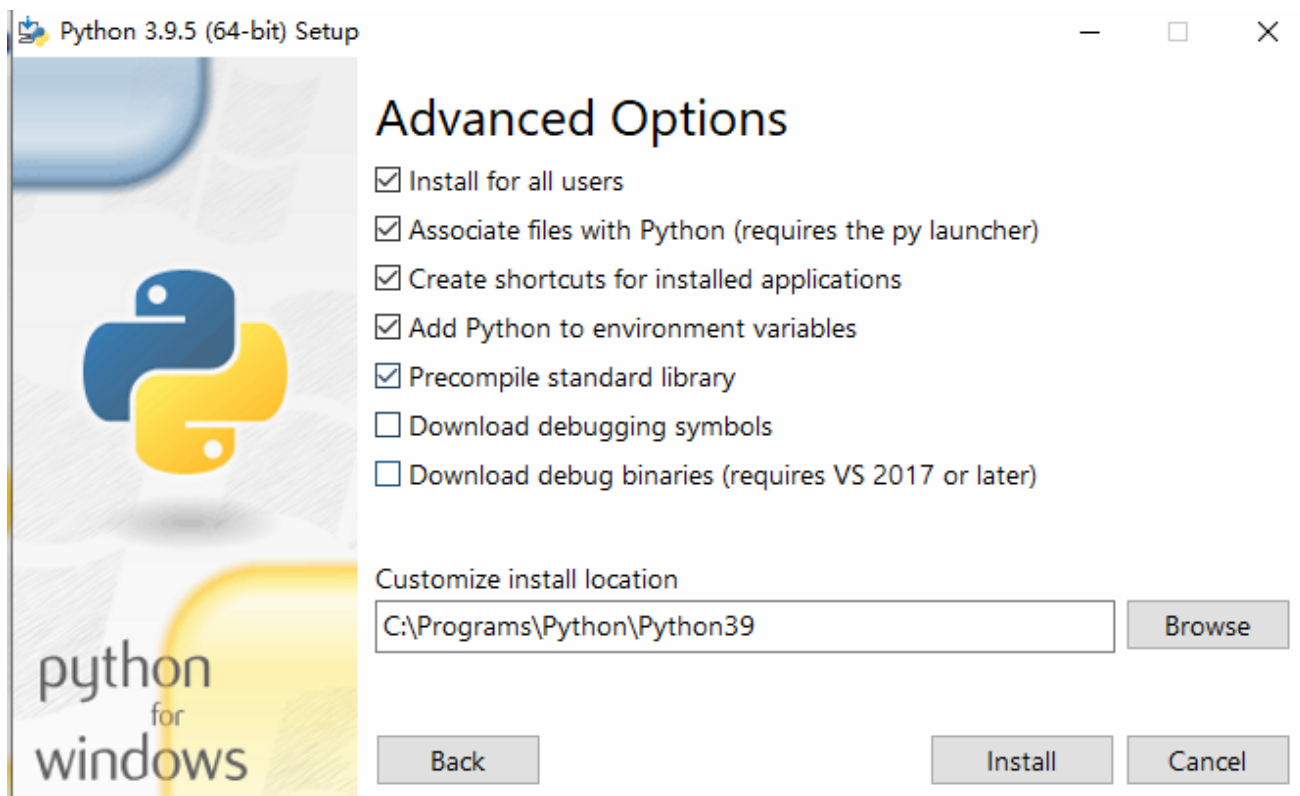
Python releases by version number:

Release version	Release date		Click for more
Python 3.12.7	Oct. 1, 2024	Download	Release Notes
Python 3.11.10	Sept. 7, 2024	Download	Release Notes
Python 3.10.15	Sept. 7, 2024	Download	Release Notes
Python 3.12.6	Sept. 6, 2024	Download	Release Notes
Python 3.9.20	Sept. 6, 2024	Download	Release Notes
Python 3.8.20	Sept. 6, 2024	Download	Release Notes
Python 3.12.5	Aug. 6, 2024	Download	Release Notes

View older releases

安装步骤:

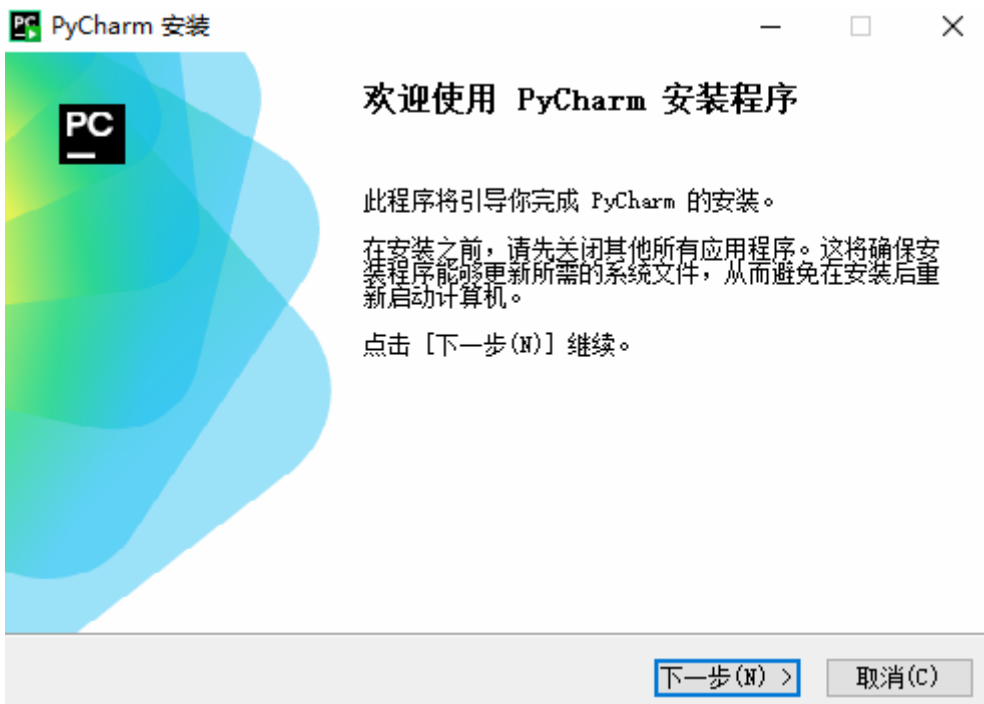




```
1 win键+r键 输入cmd
2 查看python 版本:
3 C:\Users\Administrator>python --version
4 Python 3.9.5
5 模拟下载一个库: requests
6 C:\Users\Administrator>pip install requests
7 Collecting requests
```

```
8 可以根据更新pip库
9 C:\Users\Administrator>python -m pip install --upgrade pip
10 补充小知识：由于官方的镜像源比较慢可以在将来下载不同库时，选择国内镜像源
11 举例：
12 清华大学：https://pypi.tuna.tsinghua.edu.cn/simple
13 下载：pymysql
14 C:\Users\Administrator>pip install pymysql -i https://pypi.tuna.tsinghua.edu.cn/simple
15 Looking in indexes: https://pypi.tuna.tsinghua.edu.cn/simple
16 Collecting pymysql
17   Downloading
    https://pypi.tuna.tsinghua.edu.cn/packages/0c/94/e4181a1f6286f545507528c78016e00065ea9132
    76888db2262507693ce5/PyMySQL-1.1.1-py3-none-any.whl (44 kB)
18 Installing collected packages: pymysql
19 Successfully installed pymysql-1.1.1
```

2.2 开发软件



PyCharm 安装

PC

选择安装位置

选择 PyCharm 的安装文件夹。

安装程序将把 PyCharm 安装到以下目录。要安装到另一个目录，请点击 [浏览 (B)...] 并选择其他的文件夹。 点击 [下一步 (N)] 继续。

安装目录

E:\Program Files\JetBrains\PyCharm 2024.1.4

浏览(B)...

所需空间: 2.6 GB

可用空间: 364.9 GB

< 上一步(P)

下一步(N) >

取消(C)

PyCharm 安装

PC

安装选项

配置 PyCharm 安装

创建桌面快捷方式

☒ PyCharm

更新上下文菜单

☐ 添加“将文件夹作为项目打开”

创建关联

☐ .py

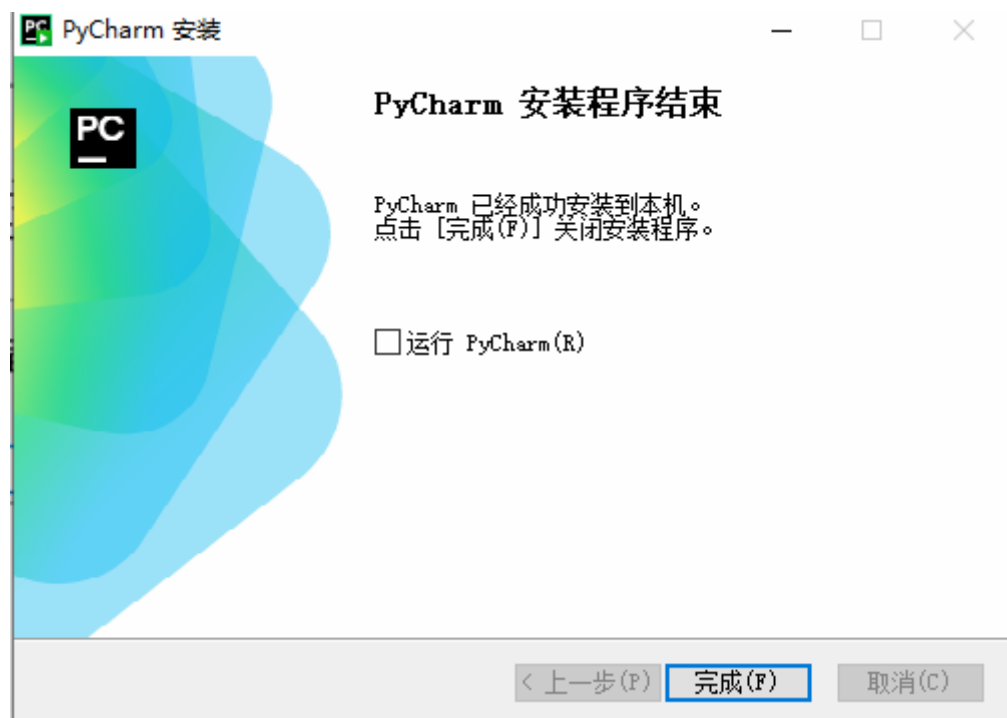
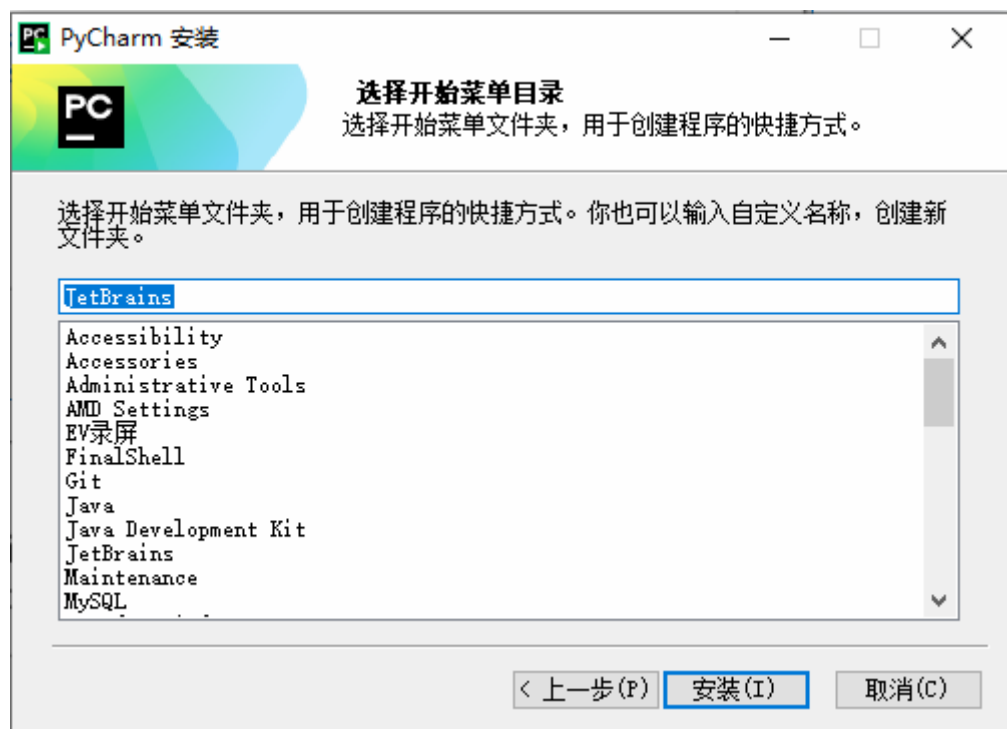
更新 PATH 变量 (需要重启)

☐ 添加“bin”文件夹到 PATH

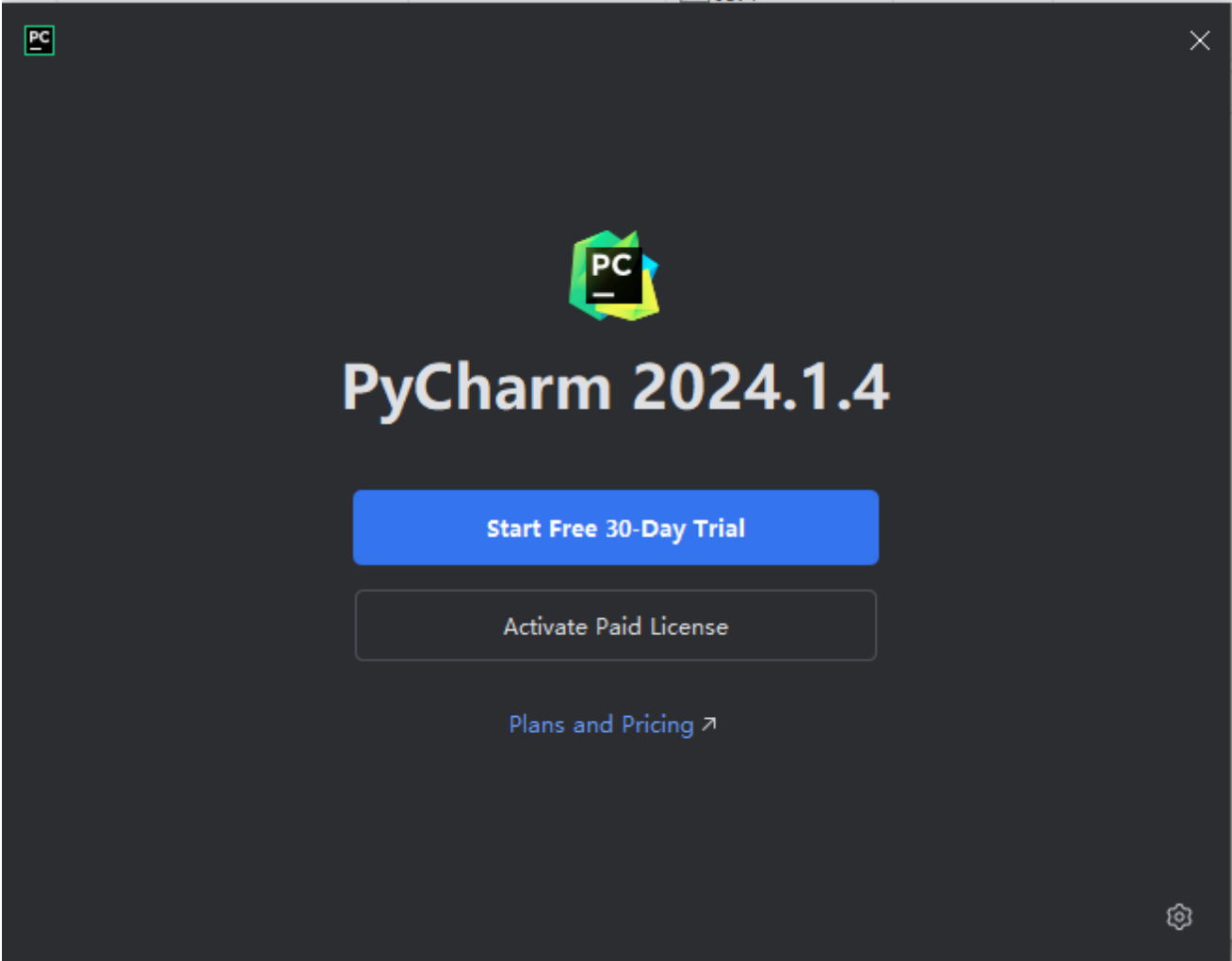
< 上一步(P)

下一步(N) >

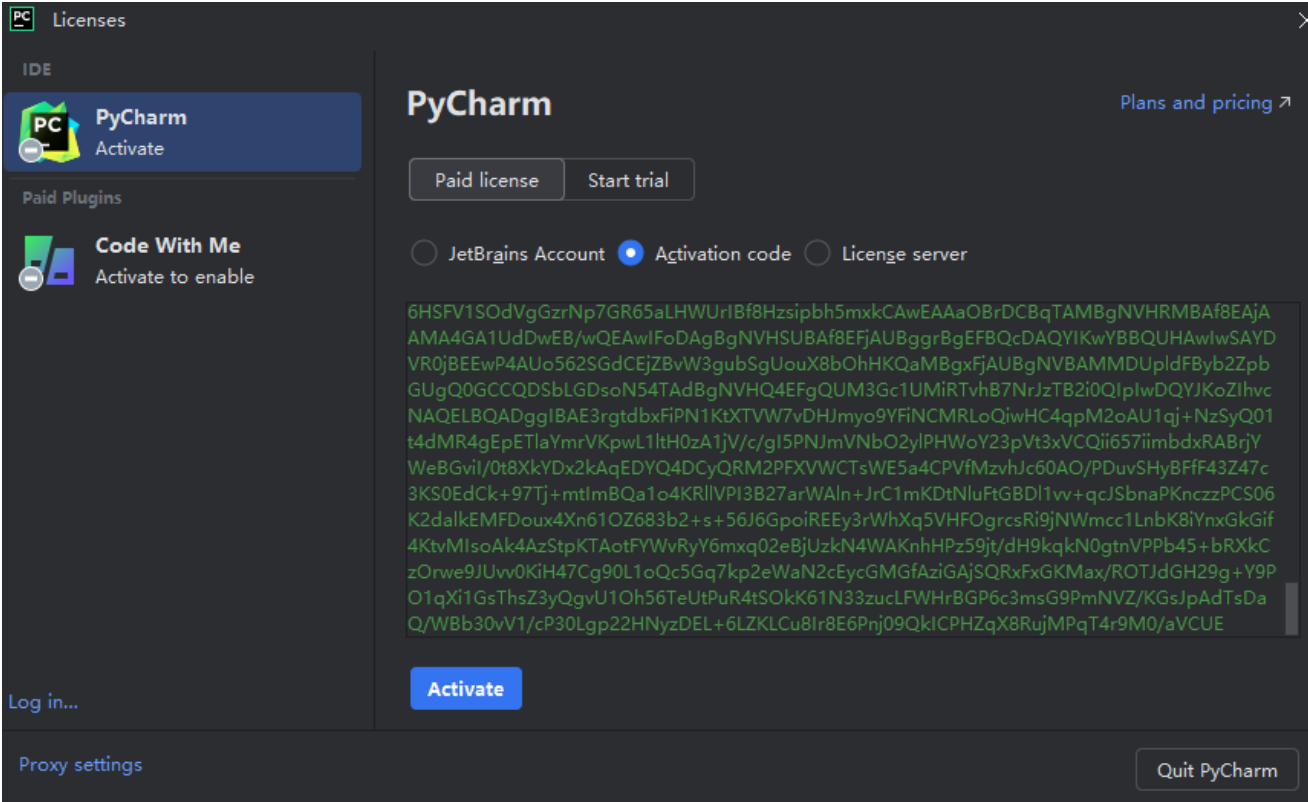
取消(C)



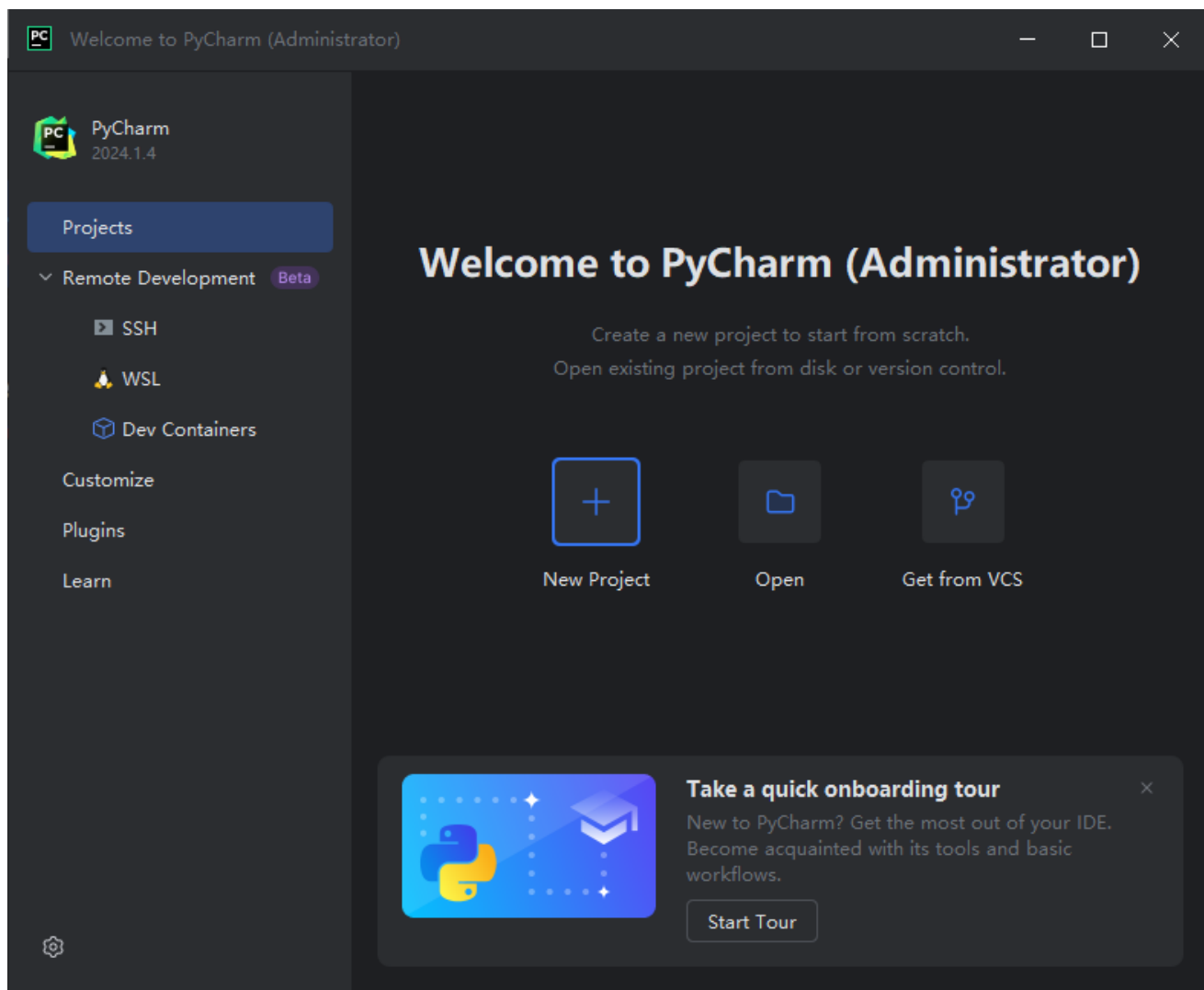
补充小知识：



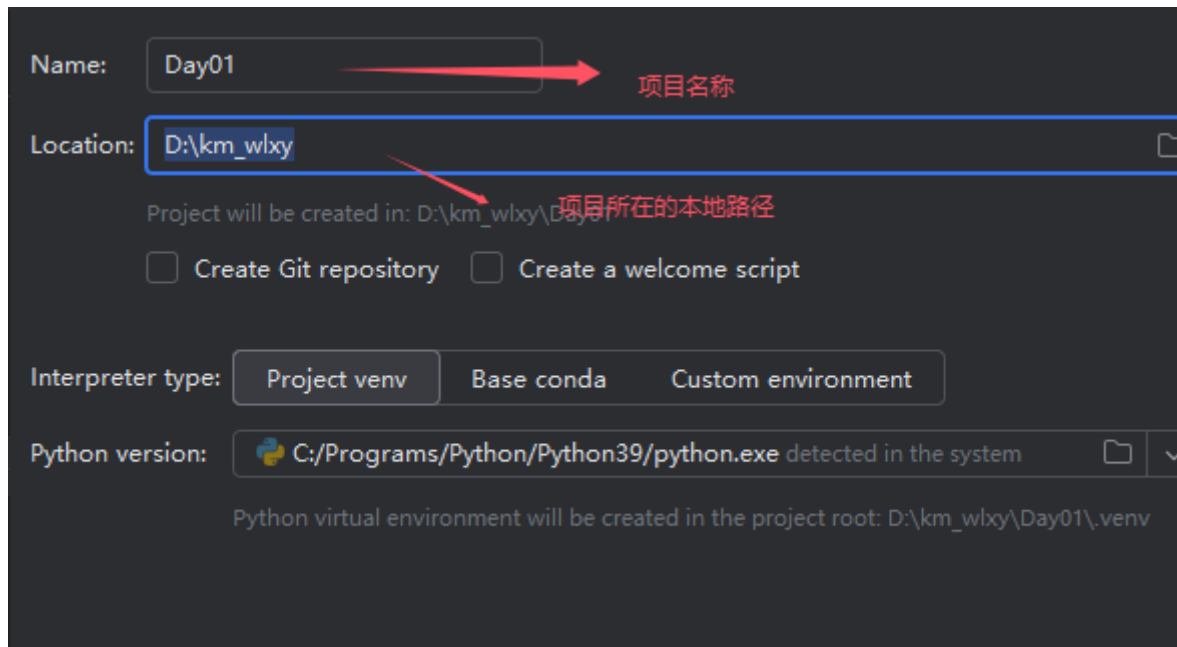
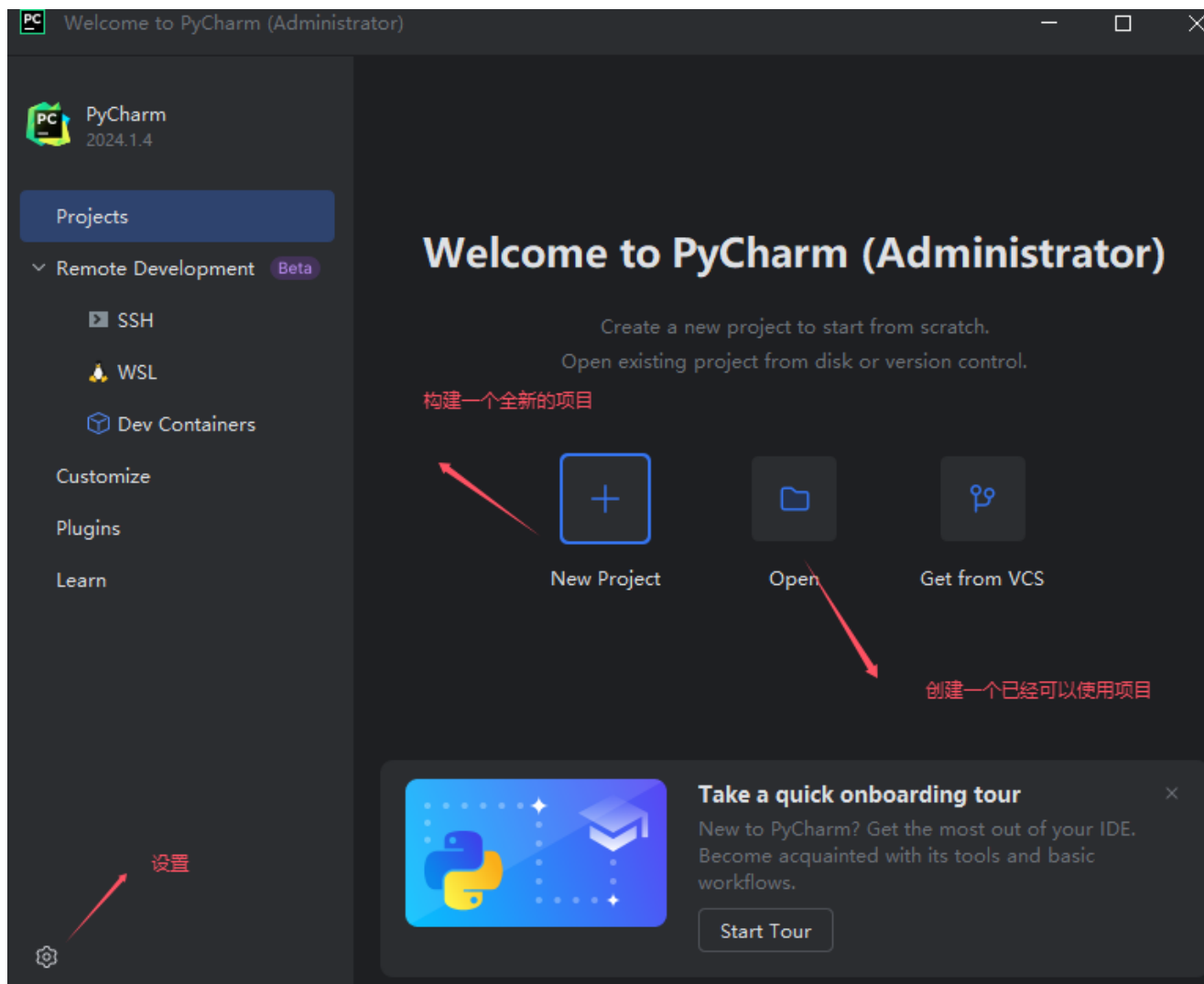
破解:

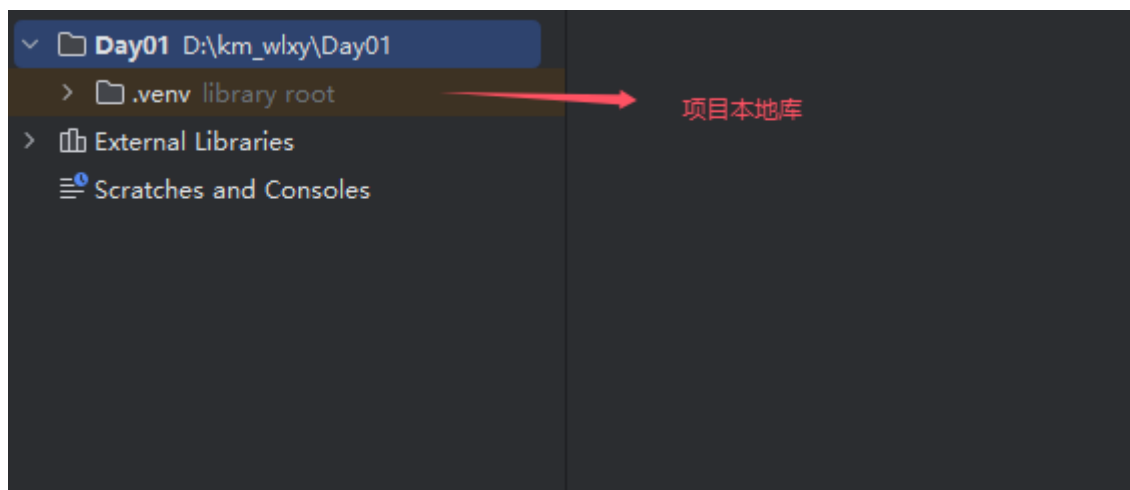


点击activate: 激活2123年

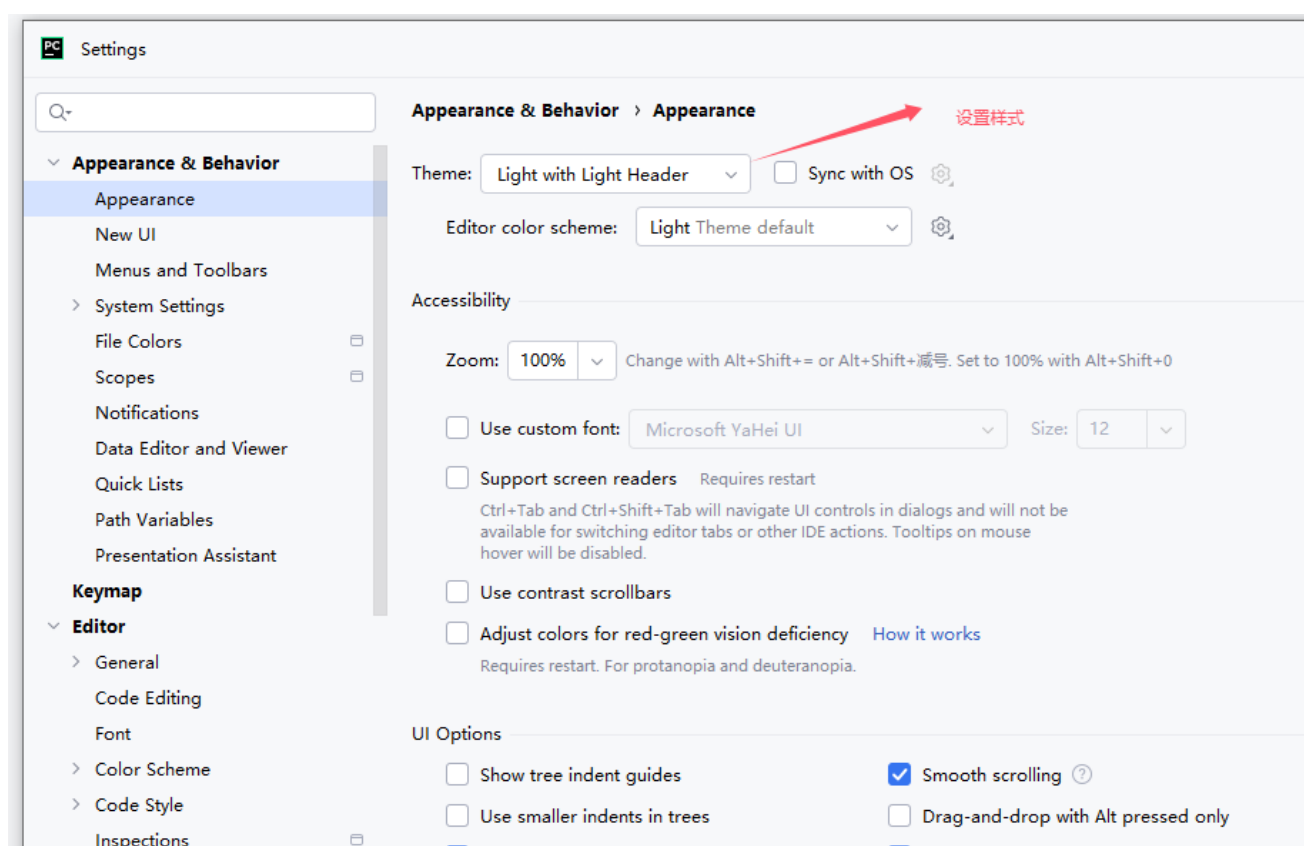


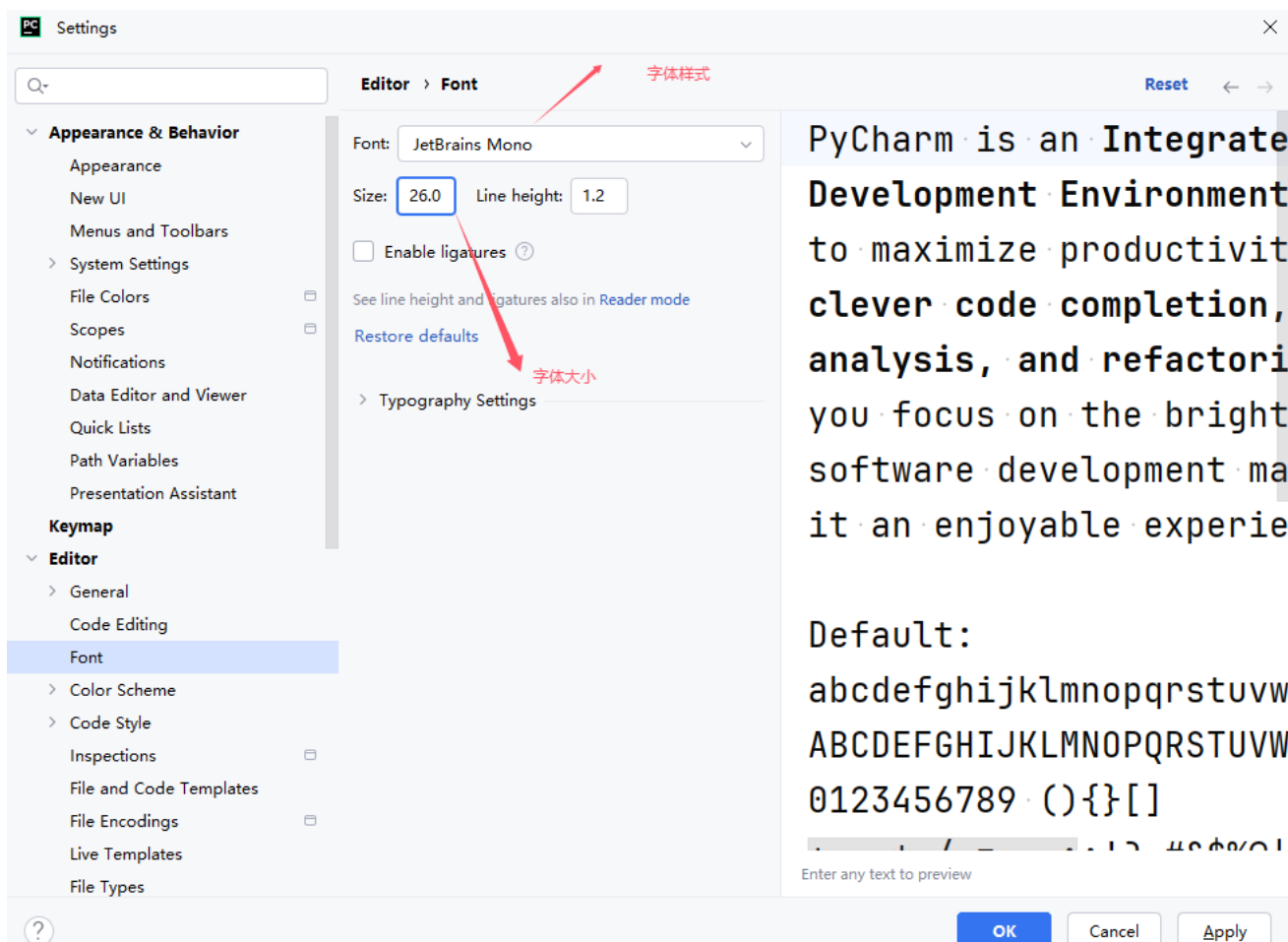
2.3 创建一个python项目





设置外观:





编写第一个python代码：

```
1 print("hello world")
2
3
4
5 if __name__ == '__main__':
6     print('hello world')
```

三、知识点讲解

3.1 python中数据类型

- 1 在python中数据类型可以分为简单数据类型：
- 2 整数数据类型
- 3 浮点数据类型
- 4 字符串数据类型
- 5 bool数据类型
- 6 """

```
7 在python注释可以分为多行注释和单行注释
8 python的基础数据类型:
9 18 <class 'int'>
10 3000.1 <class 'float'>
11 zs <class 'str'>
12 hello world <class 'str'>
13 True <class 'bool'>
14 """
15 # 单行注释, 整数类型
16 age = 18
17 # 输出当前变量age, 以及age的类型
18 print(age,type(age))
19 # 浮点数类型
20 salary = 3000.1
21 # 输出当前值为3000.1, 类型为float
22 print(salary,type(salary))
23 # 字符串类型, 注意在python中 ' ' 和 " " 作用是一样跟javascript类型
24 name = 'zs'
25 print(name,type(name))
26 say = "hello world"
27 print(say,type(say))
28 # 在python中也存在有bool类型,True 和 False
29 flag = True
30 print(flag,type(flag))
31 # 请输入学生成绩, 学生姓名, 学生的补助
32 name = input("请输入学生姓名:...")
33 print(name,type(name))
34 grade = input ("请输入学生成绩:...")
35 print(grade,type(grade))
36 # 把成绩转换为int类型
37 grade_int = int(grade)
38 print(grade_int,type(grade_int))
39 money = input("请输入学生补助: ...")
40 money_float = float(money)
41 print(money_float,type(money_float))
42 print(money,type(money))
43
44 my_age =18
45 print(my_age,type(my_age))
46 str_age = str(my_age)
47 print(str_age,type(str_age))
48 # -----类型转换类似于--java中---强制类型转换  double b =18.8  int b =(int)b
```

```
49 复杂类型：
50 # 在python中列表，list，可以存储不同数据类型数据，有序，可重复
51 my_list = [1,2,3,4,5,5,4,3,2,1,'hello world']
52 print(my_list,my_list[0])
53 # 在列表中添加一个元素
54 my_list.append('hello china') # 在列表末尾处添加一个元素
55 print(my_list)
56 my_list.insert(2,'hello 昆明')
57 print(my_list)
58 # 查询，在列表中进行查询，主要通过下标来查询，下标从0开始，如果要从最后一个元素开始查询-1,倒数第二-2
59 print(my_list[-1],my_list[-2],my_list[2])
60 # 通过切片方式查询
61 print(my_list[:])
62 # 左闭右开
63 print(my_list[0:2])
64 print(my_list[2:3])
65 print(my_list[2:])
66 print(my_list[-2:])
67 # 删除,通过下标来删除某个元素
68 print(my_list)
69 del my_list[3]
70 print(my_list)
71 # 根据内容删除
72 my_list.remove(3)
73 print(my_list)
74 # 删除整个列表---NameError: name 'my_list' is not defined
75 # del my_list
76 # print(my_list)
77 # 修改
78 my_list[3]="hahah"
79 print(my_list)
80 # 元组：
81 # 特点:有序,可重复，允许存放不同数据类型。区别列表：只能查询，不能修改和删除
82 student_tuple = ('张三','李四','王五')
83 print(student_tuple,type(student_tuple))
84 print(student_tuple[0])
85 print(student_tuple[-1])
86 # 注意元组是可以整个删除的---NameError: name 'student_tuple' is not defined
87 del student_tuple
88 print(student_tuple)
89 # 字典
```

```

90 # 特点: 字典中元素是成对出现, cunz key:value, 多个元素是, 隔开
91 teacher_dict = {'name': '刘德华', 'age': 65, 'sex': '男'}
92 print(teacher_dict, type(teacher_dict))
93 print(teacher_dict['name'])
94 print(teacher_dict['age'])
95 print(teacher_dict['sex'])
96 # 字典中添加一个新的元素
97 teacher_dict['height'] = 172
98 print(teacher_dict)
99 # 删除字典中某个
100 del teacher_dict['sex']
101 print(teacher_dict)
102
103 # 集合 set
104 # 无序、不可以重复
105 sex_set = {'男', '男', '女', '女'}
106 print(sex_set)
107 小作业:
108 1、在控制台输入一个老师信息, 并且保存成字典, 编号, 姓名, 年龄, 性别, 学科, 工资要求年龄为 int, 工资
    为 float
109 考点:
110     a、如何定义字典类型
111     b、input的用法
112     c、类型之间转换
113     d、在字典中添加一个元素
114 # 列表推导式
115 """
116 2、已知一个列表为[1,2,3,4,5,5,4,3,2,1,'hello world'], 用已经学过知识, 输出不重复的元素 (选作)
117 """
118 asc_list = [1,2,3,4,5,5,4,3,2,1,'hello world']
119 # mid_set = set(asc_list)
120 # new_list = list(mid_set)
121 # print(new_list)
122 # 利用列表推导式
123 mid_set = { item for item in asc_list }
124 print(mid_set)
125 # 利用列表推导式
126 new_list = [ item for item in mid_set ]
127 print(new_list)

```

3.2 python中语句

- 1 在任何一门编程语言中执行顺序都是由上倒下，如果有一个需求需要改变语句的执行顺序，可以利用分支语句来改变顺序，或者循环语句来改变语句顺序。
- 2 在python中语句，分支语句和循环语句

3.2.1 分支语句

```
1 案例：
2 """
3 分支语句：
4 if 条件：
5     语句
6 else:
7     语句
8 """
9 # 分支语句，可以分为简单分支语句和复杂分支语句
10
11 # 统计分析小张同学数学有没有及格
12 grade = 59
13
14 if grade >=60:
15     print("小张及格了")
16     print('haha')
17 else:
18     print("小张没有及格")
19
20 # 请在控制台上输入学生数学成绩，然后输出学生所处的等级，优秀，良好，一般，及格，不及格
21 math_grade = int(input("请输入数学成绩：...成绩为整数"))
22
23 if math_grade >85 and math_grade <=100:
24     print('数学成绩优秀')
25 elif math_grade >80 and math_grade <=85:
26     print('数学成绩良好')
27 elif math_grade >70 and math_grade <=80:
28     print('数学成绩一般')
29 elif math_grade >60 and math_grade <=70:
30     print('数学成绩及格')
31 elif math_grade >0 and math_grade <=60:
32     print('数学成绩不及格')
```

```
33 else:
34     print('输入成绩无效')
```

3.2.2 循环语句

```
1  输出1-20之间所有数:
2  """
3  输出1-20之间所有数:
4  """
5  # 左闭右开
6  for i in range(1,21):
7      print(i)
8
9  # 遍历列表
10 my_list = [1,2,3,4,5,6,7,8,9]
11
12 for item in my_list:
13     print(item)
14
15 # 列表推导式
16 """
17 2、已知一个列表为[1,2,3,4,5,5,4,3,2,1,'hello world'],用已经学过知识，输出不重复的元素（选作）
18 """
19 asc_list = [1,2,3,4,5,5,4,3,2,1,'hello world']
20 # mid_set = set(asc_list)
21 # new_list = list(mid_set)
22 # print(new_list)
23 # 利用列表推导式
24 mid_set = { item for item in asc_list }
25 print(mid_set)
26 # 利用列表推导式
27 new_list =[ item for item in mid_set]
28 print(new_list)
```

3.3 字符串类型处理

```

1  """
2  在字符串中常用方法
3  """
4  message = '三室一厅 | 临近地铁 | 南北方向 | 2018年建 | 小区免费停车, 物业费为0'
5  # 切割字符串
6  mes_list = message.split('|')
7  # 利用for的推导式实现去除空格,strip() 去除字符串两边的空格
8  new_list = [item.strip() for item in mes_list]
9  print(new_list)
10 # 想要字符串中年份----只要年份
11 year_str = new_list[-2]
12 # 利用切片实现字符串的切割
13 year = int(year_str[:-2])
14 print(year)
15 # 字符串替换
16 result_str = new_list[-1]
17 res_str = result_str.replace('费','*')
18 print(res_str)
19 # 判断
20 print(! message.startswith('三室一厅'))
21 # 多个字符串的连接
22 a = 'hello '
23 b = 'world'
24 my_str = a.join(b)
25 print(my_str)

```

3.4 数值格式化

```

1  """
2  格式化数据
3  """
4  name = "张三"
5  age = 18
6  sex = '男'
7  salary = 5000.8
8
9  # 第一种方式通过 %s 表示字符串 %d 整型 %f 浮点数 ----%.2f 表示保留两位小数
10 message = '姓名: %s, 年龄: %d, 性别: %s, 工资: %.2f'%(name, age, name, salary)

```

```
11 print(message)
12 # 第二种方式 {} 表示占位符
13 new_message = '姓名: {},年龄: {},性别: {},工资:{}'.format(name,age,name,salary)
14 print(new_message)
```

3.5 文件读写

```
1 关于文件读写，在python语言中也支持文件读写工作。
2 open函数
3 """
4 原始文件读写
5 """
6 # mode: w 表示写  r 表示读  b 二进制  wb: 表示二进制写  rb:表示二进制读  a :追加
7 # file = open("a.txt","w",encoding="utf-8")
8 # file.write("哈哈")
9 # file.close()
10 file = open('a.txt','r',encoding='utf-8')
11 str_ = file.read()
12 print(str_)
13 file.close()
14 注意：在使用open函数一定要进行关闭，一般不推荐
15 with open("b.txt",'w',encoding='utf-8') as f:
16     f.write("hello wlxy")
17
18 with open("b.txt",'r',encoding='utf-8') as f:
19     str_ = f.read()
20     print(str_)
```

3.6 爬虫

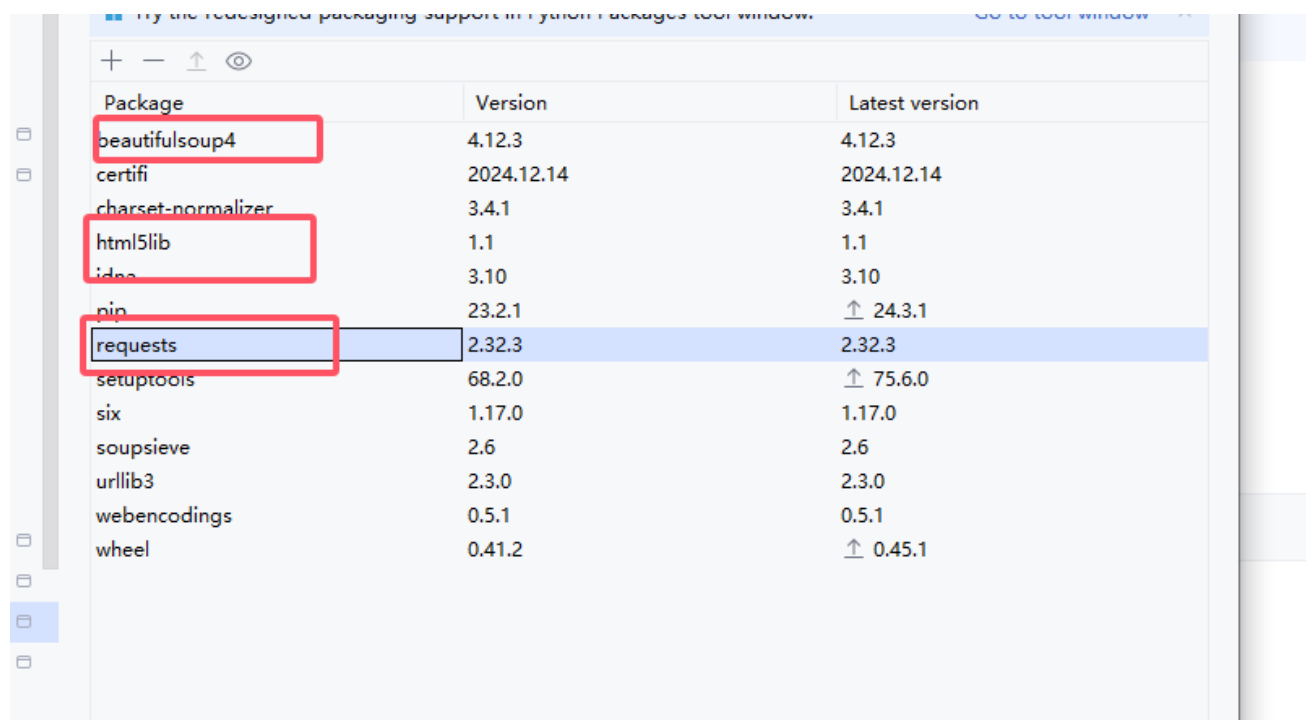
3.6.1 爬虫概述

- 1 什么叫爬虫？
- 2 互联网中有很多信息，用户可以通过访问浏览器方式来请求一个网页，获取一个网页的数据，爬虫本质利用代码模拟浏览器行为，爬取网上资源
- 3 上网？
- 4 在浏览器上输入一个网址，按住回车，发送一个请求，网站会返回响应----网页

- 5 爬虫----发送一个请求-----服务返回一个响应（网页中html页面，获取html页面某个标签中值或者属性值）-----相关解析网页技术---解析对应的
- 6 网页----获取想要的值
- 7 为什么，获取数据过程叫做爬虫？
- 8 爬虫行为比较像蜘蛛捕食

3.6.2 爬虫准备工作

- 1 在python中，爬虫技术有很多，本次采用最原始爬虫
- 2 requests
- 3 解析方式:beautifulsoup4----html5lib
- 4 下载对应依赖:
- 5 (.venv) PS D:\km_wlxy\Day02> pip install requests
- 6 (.venv) PS D:\km_wlxy\Day02> pip install beautifulsoup4
- 7 (.venv) PS D:\km_wlxy\Day02> pip install html5lib



Package	Version	Latest version
beautifulsoup4	4.12.3	4.12.3
certifi	2024.12.14	2024.12.14
charset-normalizer	3.4.1	3.4.1
html5lib	1.1	1.1
idna	3.10	3.10
pip	23.2.1	24.3.1
requests	2.32.3	2.32.3
setuptools	68.2.0	75.6.0
six	1.17.0	1.17.0
soupsieve	2.6	2.6
urllib3	2.3.0	2.3.0
webencodings	0.5.1	0.5.1
wheel	0.41.2	0.45.1

3.6.3 站长之家

- 1 需求:
- 2 访问站长之家，爬取站长之家图片数据，下载到本地
- 3 思路:
- 4 先爬取一页数据:

```
5  """
6  爬虫:
7  a、导入相关库
8  b、定义爬虫网站
9  c、模拟浏览器发送一个请求
10 d、创建一个解析响应的对象
11 e、写入本地，或者输出
12 """
13 import requests # 请求库
14 from bs4 import BeautifulSoup # 解析网页响应的库
15 import os
16
17 dir = 'tupian'
18 # 如果文件夹不存在则创建文件夹
19 if not os.path.exists(dir):
20     os.makedirs(dir)
21     print("创建一个文件夹")
22
23
24 # 爬取的网址
25 url = 'https://sc.chinaz.com/tupian/'
26 # 模拟浏览器
27 header = {
28     "User-Agent":
29     "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
    Chrome/118.0.0.0 Safari/537.36"
30 }
31 # 发送请求
32 response = requests.get(url, headers=header)
33 # 检查请求是否响应成功 200 表示成功响应 300 权限 400 404 网址不对 403 封号 418 非正常请
    求
34 # 创建解析响应的对象
35 bs = BeautifulSoup(response.content, 'html5lib')
36 # 获取图片地址信息 find() 返回第一个元素 find_all() 返回所有元素
37 img_list = bs.find_all('img', attrs={'class': 'lazy'})
38 # 测试img_list是否查找到
39 for img in img_list:
40     # img_url = img['data-original']
41     # 通过属性名获取属性值
42     img_url = "https:{}".format(img.get('data-original'))
43     # 下载本地
44     with open("{}{}".format(dir, img_url.split("/")[-1]), "wb") as f:
45         f.write(requests.get(img_url, headers=header).content)
```

```
46         print("写出一张图片成功")
47 总结爬取多页数据:
48 """
49 爬虫:
50 a、导入相关库
51 b、定义爬虫网站
52 c、模拟浏览器发送一个请求
53 d、创建一个解析响应的对象
54 e、写入本地，或者输出
55 """
56 import requests # 请求库
57 from bs4 import BeautifulSoup # 解析网页响应的库
58 import os
59
60 dir = 'tupian'
61 # 如果文件夹不存在则创建文件夹
62 if not os.path.exists(dir):
63     os.makedirs(dir)
64     print("创建一个文件夹")
65
66
67 # 爬取的网址
68 url = 'https://sc.chinaz.com/tupian/'
69
70 """
71 下载图片
72 """
73 def download(url):
74     # 模拟浏览器
75     header = {
76         "User-Agent":
77             "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
78             Gecko) Chrome/118.0.0.0 Safari/537.36"
79     }
80     # 发送请求
81     response = requests.get(url, headers=header)
82     # 检查请求是否响应成功 200 表示成功响应 300 权限 400 404 网址不对 403 封号 418 非正
83     常请求
84     # 创建解析响应的对象
85     bs = BeautifulSoup(response.content, 'html5lib')
86     # 获取图片地址信息 find() 返回第一个元素 find_all() 返回所有元素
87     img_list = bs.find_all('img', attrs={'class': 'lazy'})
88     # 测试img_list是否查找到
```

```

87     for img in img_list:
88         # img_url = img['data-original']
89         # 通过属性名获取属性值
90         img_url = "https:{}".format(img.get('data-original'))
91         # 下载本地
92         with open("{}{}".format(dir, img_url.split("/")[-1]), "wb") as f:
93             f.write(requests.get(img_url, headers=header).content)
94             print("写出一张图片成功")
95
96 if __name__ == '__main__':
97     base_url="https://sc.chinaz.com/tupian/"
98     url = ''
99     for i in range(1,6):
100         if i == 1:
101             url = base_url
102         else:
103             url = "{}index{}.html".format(base_url, i)
104         download(url)
105 作业：
106 爬取站长之家：https://sc.chinaz.com/tupian/fengjing.html
107 爬取5页
108 思路：
109 参考上述案例

```

3.6.4 数据----来源

- 1 通常来实现数据分析的案例，数据来源一般可以分为三种：
- 2 a、自有数据
- 3 b、网上爬虫技术，爬取相关的网站获取相关的数据
- 4 c、公开的数据源----阿里天池数据源
- 5 本次项目中所有采用数据为：
- 6 豆瓣电影的数据

3.6.5 豆瓣Top250数据爬取

- 1 思路：
- 2 1、导入相关的库

```
3     2、定义爬取网站
4     3、获取网页解析的对象Beautifulsoup
5     4、解析网页
6     5、写入本地----CSV
7 步骤1:
8     爬取一页数据
9     ""
10 爬取豆瓣电影top250电影信息
11 ""
12 import requests
13 from bs4 import BeautifulSoup
14 import re
15 import csv
16 import os
17
18 # 定义一个存储文件信息的文件夹
19 project_path = os.path.dirname(__file__)
20 dir = os.path.join(project_path, "douban")
21
22 # 判断该文件夹是否存在，如果不存在则创建文件夹
23 if not os.path.exists(dir):
24     # 创建文件夹
25     os.makedirs(dir)
26
27 # 定义请求头，模拟浏览器行为
28 headers={
29     'User-Agent':
30     'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
31     Chrome/118.0.0.0 Safari/537.36'
32 }
33 # 定义爬取top250网址
34 url = 'https://movie.douban.com/top250'
35
36 # 发送请求返回响应
37 response = requests.get(url, headers=headers)
38
39 # 测试响应是否成功
40 # print(response.status_code)
41
42 # 创建一个解析网页的对象
43 bs = BeautifulSoup(response.content, 'html5lib')
```

```
44
45 # 解析网页获取对应数据
46 div_list = bs.find_all('div',attrs={"class":'item'})
47 # 遍历div列表
48 for div in div_list:
49     # 电影图片地址
50     img_url = div.find("img").get("src")
51     # 获取电影名称
52     hd_mes = div.find("div",attrs={"class":'hd'}).text.replace(" ", "").replace("\n", "")
53     title = hd_mes.split("/")[0].strip()
54     # 获取导演等相关信息
55     bd_mes = div.find("div",attrs={"class":'bd'}).text.strip().split("\n")
56     bd_new= [item.strip().replace("\xa0", "") for item in bd_mes]
57     dy = bd_new[0].split(":")[1].split(" ")[1]
58     # 上映时间
59     start_time =bd_new[1].split("/")[0]
60     # 地址
61     addr = bd_new[1].split("/")[1]
62     # 电影类型
63     type = bd_new[1].split("/")[2].split(" ")[0]
64     # 评分
65     grade = bd_new[7]
66     # 评论人数
67     rating_count = bd_new[9][: -3]
68     # 评论内容
69     rating_mes = bd_new[13]
70     # 定义一个列表
71     message=[title,img_url,dy,start_time,addr,type,grade,rating_count,rating_mes]
72     print(message)
73
74 步骤2:
75     爬取10页的数据
76     """
77 爬取豆瓣电影top250电影信息
78     """
79 import requests
80 from bs4 import BeautifulSoup
81 import re
82 import csv
83 import os
84
85 # 定义一个存储文件信息的文件夹
```

```
86 project_path = os.path.dirname(__file__)
87 dir = os.path.join(project_path, "douban")
88
89 # 判断该文件夹是否存在，如果不存在则创建文件夹
90 if not os.path.exists(dir):
91     # 创建文件夹
92     os.makedirs(dir)
93
94 # 定义请求头，模拟浏览器行为
95 headers={
96     'User-Agent':
97     'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
98     Chrome/118.0.0.0 Safari/537.36'
99 }
100
101 def get_html(url):
102     # 发送请求返回响应
103     response = requests.get(url, headers=headers)
104     # 测试响应是否成功
105     # print(response.status_code)
106     # 创建一个解析网页的对象
107     bs = BeautifulSoup(response.content, 'html5lib')
108     print(response.status_code)
109     # 解析网页获取对应数据
110     div_list = bs.find_all('div', attrs={"class": 'item'})
111     # 遍历div列表
112     for div in div_list:
113         # 电影图片地址
114         img_url = div.find("img").get("src")
115         # 获取电影名称
116         hd_mes = div.find("div", attrs={"class": 'hd'}).text.replace(" ",
117         "").replace("\n", "")
118         title = hd_mes.split("/")[0].strip()
119         # 获取导演等相关信息
120         bd_mes = div.find("div", attrs={"class": 'bd'}).text.strip().split("\n")
121         bd_new = [item.strip().replace("\xa0", "") for item in bd_mes]
122         dy = bd_new[0].split(":")[1].split(" ")[1]
123         # 上映时间
124         start_time = bd_new[1].split("/")[0]
125         # 地址
126         addr = bd_new[1].split("/")[1]
127         # 电影类型
128         type = bd_new[1].split("/")[2].split(" ")[0]
129         # 评分
```

```

127     grade = bd_new[7]
128     # 评论人数
129     rating_count = bd_new[9][:3]
130     rating_mes=''
131     # 评论内容
132     if len(bd_new) <= 13:
133         rating_mes = "哈哈"
134     else:
135         rating_mes = bd_new[13]
136     # 定义一个列表
137     message = [title, img_url, dy, start_time, addr, type, grade, rating_count,
rating_mes]
138     # 写入到本地文件中
139     with open('{}\douban.csv'.format(dir), 'a', newline='') as f:
140         # 获取csv写出对象
141         csv_writer = csv.writer(f)
142         # 写出数据信息
143         csv_writer.writerow(message)
144     print("写出一页")
145
146 if __name__ == '__main__':
147     # 定义爬取top250网址
148     base_url = 'https://movie.douban.com/top250'
149     """
150     第一页地址:
151     # 定义爬取top250网址
152     url = 'https://movie.douban.com/top250'
153     第二页地址:
154     https://movie.douban.com/top250?start=25&filter=
155     第三页地址:
156     https://movie.douban.com/top250?start=50&filter=
157     第十页地址:
158     https://movie.douban.com/top250?start=225&filter=
159     """
160     url = ''
161     for i in range(1,11):
162         if i ==1:
163             url = base_url
164         else:
165             url = "{}?start={}&filter=".format(base_url, (i-1)*25)
166         get_html(url)
167     编码:

```

```
168 创建一个Day03项目，安装相关的库
169 (.env) PS D:\km_wlxy\Day03> pip install requests
170 (.env) PS D:\km_wlxy\Day03> pip install beautifulsoup4
171 (.env) PS D:\km_wlxy\Day03> pip install html5lib
172 爬取豆瓣：如果不加编码则出现问题：
173 """
174 爬取豆瓣电影top250电影信息
175 """
176 import requests
177 from bs4 import BeautifulSoup
178 import re
179 import csv
180 import os
181
182 # 定义一个存储文件信息的文件夹
183 project_path = os.path.dirname(__file__)
184 dir = os.path.join(project_path, "douban")
185
186 # 判断该文件夹是否存在，如果不存在则创建文件夹
187 if not os.path.exists(dir):
188     # 创建文件夹
189     os.makedirs(dir)
190
191 # 定义请求头，模拟浏览器行为
192 headers={
193     'User-Agent':
194     'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
195     Chrome/118.0.0.0 Safari/537.36'
196 }
197
198 def get_html(url):
199     # 发送请求返回响应
200     response = requests.get(url, headers=headers)
201     # 测试响应是否成功
202     # print(response.status_code)
203     # 创建一个解析网页的对象
204     bs = BeautifulSoup(response.content, 'html5lib')
205     print(response.status_code)
206     # 解析网页获取对应数据
207     div_list = bs.find_all('div', attrs={"class": 'item'})
208     # 遍历div列表
209     for div in div_list:
210         # 电影图片地址
```

```
209     img_url = div.find("img").get("src")
210     # 获取电影名称
211     hd_mes = div.find("div", attrs={"class": 'hd'}).text.replace(" ",
""").replace("\n", "")
212     title = hd_mes.split("/")[0].strip()
213     # 获取导演等相关信息
214     bd_mes = div.find("div", attrs={"class": 'bd'}).text.strip().split("\n")
215     bd_new = [item.strip().replace("\xa0", "") for item in bd_mes]
216     dy = bd_new[0].split(":")[1].split(" ")[1]
217     # 上映时间
218     start_time = bd_new[1].split("/")[0]
219     # 地址
220     addr = bd_new[1].split("/")[1]
221     # 电影类型
222     type = bd_new[1].split("/")[2].split(" ")[0]
223     # 评分
224     grade = bd_new[7]
225     # 评论人数
226     rating_count = bd_new[9][:3]
227     rating_mes=''
228     # 评论内容
229     if len(bd_new) <= 13:
230         rating_mes = "哈哈"
231     else:
232         rating_mes = bd_new[13]
233     # 定义一个列表
234     message = [title, img_url, dy, start_time, addr, type, grade, rating_count,
rating_mes]
235     # 写入到本地文件中
236     with open('{}\douban.csv'.format(dir), 'a',encoding='utf-8',newline='') as f:
237         # 获取csv写出对象
238         csv_writer = csv.writer(f)
239         # 写出数据信息
240         csv_writer.writerow(message)
241     print("写出一页")
242
243 if __name__ == '__main__':
244     # 定义爬取top250网址
245     base_url = 'https://movie.douban.com/top250'
246     """"
247     第一页地址:
248     # 定义爬取top250网址
249     url = 'https://movie.douban.com/top250'
```

```
250     第二页地址：
251     https://movie.douban.com/top250?start=25&filter=
252     第三页地址：
253     https://movie.douban.com/top250?start=50&filter=
254     第十页地址：
255     https://movie.douban.com/top250?start=225&filter=
256     ""
257     url = ''
258     for i in range(1,11):
259         if i ==1:
260             url = base_url
261         else:
262             url = "{}?start={}&filter=".format(base_url, (i-1)*25)
263         get_html(url)
```

3.7 Pandas-数据分析

3.7.1 pandas数据分析简介

- ¹ Pandas是一个强大的分析结构数据的工具集，基础是Numpy（提供高性能的矩阵运算），常用于数据的玩具和数据分析，同时也提供数据清洗功能。
- ² Pandas是python中很担心分析库，提供快速、灵活的数据结构，Pandas成为python中最好的数据分析工具。

3.7.2 Pandas官网和安装

- ¹ 官网地址：
- ²
- ³ 安装：
- ⁴ `pip install pandas`

3.7.3 pandas简单应用

- ¹ 推荐：<https://www.runoob.com/pandas/pandas-tutorial.html> ----菜鸟教程
- ² Pandas 是 Python 语言的一个扩展程序库，用于数据分析。

- 3 **Pandas** 名字衍生自术语 "panel data"（面板数据）和 "Python data analysis"（Python 数据分析）
- 4 **Pandas** 是一个开放源码、BSD 许可的库，提供高性能、易于使用的数据结构和数据分析工具
- 5 **Pandas** 一个强大的分析结构化数据的工具集，基础是 **Numpy**（提供高性能的矩阵运算）
- 6 **Pandas** 可以从各种文件格式比如 **CSV**、**JSON**、**SQL**、**Microsoft Excel** 导入数据。
- 7 **Pandas** 可以对各种数据进行运算操作，比如归并、再成形、选择，还有数据清洗和数据加工特征。
- 8 **Pandas** 广泛应用在学术、金融、统计学等各个数据分析领域。
- 9 应用方法：
- 10 数据清洗：处理缺失数据、重复数据等。
- 11 数据转换：改变数据的形状、结构或格式。
- 12 数据分析：进行统计分析、聚合、分组等。
- 13 数据可视化：通过整合 **Matplotlib** 和 **Seaborn** 等库，可以进行数据可视化。

3.7.4 Pandas的数据结构

- 1 **Pandas** 的主要数据结构是 **Series**（一维数据）与 **DataFrame**（二维数据）。
- 2 **Series**
- 3 是一种类似于 一维数组 的对象，它由一组数据（各种 **Numpy** 数据类型）以及一组与之相关的数据标签（即索引）组成。
- 4 **DataFrame**
- 5 是一个表格型的数据结构，它含有一组有序的列，每列可以是不同的值类型（数值、字符串、布尔型值）。
DataFrame 既有行索引也有列索引，它可以被看做由 **Series** 组成的字典（共同用一个索引）。
- 6 案例1: **Series**的用法
- 7 `import pandas as pd`
- 8 `# 定义列表`
- 9 `my_list=[1,2,3,4,5]`
- 10 `# 转换为pandas中Series`
- 11 `my_series=pd.Series(my_list)`
- 12 `print(my_series)`
- 13 `# 获取Series中的索引`
- 14 `index = my_series.index`
- 15 `print(index)`
- 16 `# 获取Series的值`
- 17 `values = my_series.values`
- 18 `print(values)`
- 19 `# 自定义Series`
- 20 `x = pd.Series(`
- 21 `data=[1,2,1,6,2,3,4,5],`
- 22 `index=['a','b','c','d','e','f','g','h'],`
- 23 `name='x'`
- 24 `)`

```
25 print(x)
26 # 根据索引找到值
27 print(x['c'])
28 # 取b 到 d的值
29 print(x['b':'d'])
30 # 给Series添加数据 append(无效)
31 y = pd.Series(data=[5,6,7,8],index=['f','g','h','j'])
32 x['f']=10
33 print(x)
34 # 根据索引删除对应元素
35 x = x.drop('f')
36 print(x)
37 # 有head来查询Series----tail()
38 # 查询开始两条数据
39 print(x.head(2))
40 # 查询末尾处的两条数据
41 print(x.tail(2))
42 # 根据索引进行排序,ascending指的是排序的规则, 如果False 降序 True 升序
43 x = x.sort_index(ascending=False)
44 print(x)
45 # 根据值进行排序
46 x = x.sort_values(ascending=False)
47 print(x)
48
49 print(type(x.values))
50 print(type(x.to_list()))
51 my_list = x.to_list()
52 print(my_list[0])
53
54 # map 用到函数时匿名函数
55
56 x = x.map(lambda a:a*2)
57 print(x)
58 # 变形
59 x = x.map({4:"男",2:"女"})
60 print(x)
61
62
63 def test(x):
64     return x*2
65
66 案例2: DataFrame案例:
```

67 **DataFrame** 是 **Pandas** 中的另一个核心数据结构，类似于一个二维的表格或数据库中的数据表。

68 **DataFrame** 是一个表格型的数据结构，它含有一组有序的列，每列可以是不同的值类型（数值、字符串、布尔型值）。

69 **DataFrame** 既有行索引也有列索引，它可以被看做由 **Series** 组成的字典（共用一个索引）。

70 **DataFrame** 提供了各种功能来进行数据访问、筛选、分割、合并、重塑、聚合以及转换等操作。

71 **DataFrame** 是一个非常灵活且强大的数据结构，广泛用于数据分析、清洗、转换、可视化等任务。

72 二维结构：

73 **DataFrame** 是一个二维表格，可以被看作是一个 **Excel** 电子表格或 **SQL** 表，具有行和列。可以将其视为多个 **Series** 对象组成的字典。

```
74 import pandas as pd
75 from pandas import DataFrame, Series
76
77 name_list = ['zs', 'ls', 'ww']
78 age_list = [18, 19, 20]
79 sex_list = ['0', '1', '0']
80 # 自定义DataFrame
81 df = DataFrame(
82     data={
83         'name': name_list,
84         'age': age_list,
85         'sex': sex_list
86     }
87 )
88 print(df)
89 print(df.index)
90 print(df.columns)
91 print(df.dtypes)
92 print(df.values)
93 # 取某一列值
94 print(df['name'])
95 # 使用切片---行
96 print(df[1:])
97 # 取多列----列
98 print(df[['name', 'sex']])
```

3.7.5 loc 和 iloc 的使用

```
1 pandas中可以筛选行和列方法loc和iloc
2 from pandas import DataFrame, Series
3
```

```

4 # loc和iloc都可以进行筛选
5 # 区别loc使用列进行检索，iloc使用列索引进行检索
6 # 自定义DataFrame
7 df = DataFrame(
8     data=[
9         {'name':'张三','age':18,'sex':'男'},
10        {'name':'李三','age':19,'sex':'男'},
11        {'name':'王三','age':28,'sex':'男'}
12    ],
13    index=['a','b','c']
14 )
15 # loc用法，筛选name列中所有数据
16 print(df.loc[:,['name']])
17 print(df['name'])
18 print(df.loc['a',['name','age','sex']])
19 print(df['a':'a'][['name','age','sex']])
20 # iloc
21 print(df.iloc[:,0:])
22 print(df.iloc[0:1,1:2])
23
24 小作业：
25 分别用loc和iloc方式取出 第一行 age 和sex列

```

3.7.5 pandas读写csv文件

```

1 from pandas import DataFrame, Series
2 import pandas as pd
3
4 # 1、读取csv文件
5 df = pd.read_csv("douban/douban.csv", header=None,
6                 names=['title', 'img_url', 'dy', 'start_time', 'addr', 'type',
7                       'grade', 'rating_count', 'rating_mes'])
8 # 2、输出所有列
9 print(df.columns)
10 # 3、输出所有的value值
11 # print(df.values)
12 # 4、去除空数据
13 df = df.dropna()
14 # print(df)

```

```

15 # 5、给空字段填充值
16 df = df.fillna("0")
17 print(df)
18 # 6、筛选有用数据
19 df_new = df.loc[:,['title', 'grade','rating_count']]
20 print(df_new)
21 # 7、写入本地
22 df_new.to_csv("douban/movie_grade_count.csv", index=False,header=False)
23 小作业：
24 读取爬虫豆瓣.csv文件，获取其中title，grade这两列，写入到新的csv文件中 news.csv

```

3.7.6 pandas的分组和聚合运算

```

1 准备工作：
2  """
3  改变douban.csv文件中内容
4  """
5  import pandas as pd
6
7  # 读取douban.csv文件
8  df = pd.read_csv('douban/douban.csv',names=['title', 'img_url', 'dy', 'start_time',
9  'addr', 'type',
10              'grade', 'rating_count', 'rating_mes'])
11 # head函数默认展示前五条数据
12 print(df.head())
13 # 针对于列进行变形
14 df['addr']=df['addr'].map(lambda x: x.split(" ")[0])
15 df['start_time']=df['start_time'].map(lambda x: x[0:4])
16 print(df['addr'])
17 # 给空字段添加默认值
18 df.fillna("中国")
19 # 分组统计，各个国家在top250，出现次数
20 result = df.groupby('addr').size().reset_index(name='count')
21 # 排序---ascending=False 降序
22 result =result[1:].sort_values(by=['count'], ascending=False)
23 print(result)
24
25 result.to_csv('douban/addr.csv',index=False,header=False)
26 # 分组统计，各个时间段，top250，出现次数
27 result1 = df.groupby('start_time').size().reset_index(name='count')

```

```

27 result1 = result1[0:]
28 # 排序
29 result1 = result1.sort_values(by=['count'], ascending=False)
30
31 result1.to_csv('douban/start_time.csv',index=False,header=False)
32
33 # 根据电影名称分组
34 result2 = df.groupby('title').size().reset_index(name='count')
35 result2 = result2[0:].sort_index()
36 print(result2)
37 result2.to_csv('douban/title.csv',index=False,header=False)
38 小作业：
39 根据类型进行分组，然后根据值进行排序，写入本地
40 # 聚合函数
41 """
42 聚合函数一般都是与分组函数一起使用
43 """
44 import pandas as pd
45
46 # 读取文件
47 df = pd.read_csv('douban/douban.csv',names=['title', 'img_url', 'dy', 'start_time',
48 'addr', 'type',
49 'grade', 'rating_count', 'rating_mes'])
50
51 # 统计分析，top250 各地区电影的平均评分
52 df['addr']=df['addr'].map(lambda x: x.split(" ")[0])
53 df['start_time']=df['start_time'].map(lambda x: x[0:4])
54 result = df.groupby('addr')['grade'].mean().reset_index()
55 print(result)
56 # 统计分析，各个时间段的平均评分
57 result1 = df.groupby('start_time')['grade'].mean().reset_index()
58 print(result1)
59 #agg() 合并
60 result2 = df.groupby('addr').agg({'grade':['mean','sum'],'rating_count':['max','min']})
61 print(result2)

```

3.8 数据可视化

¹ Matplotlib是python中绘图工具，一般进行数据分析的结果都可以使用图表方式进行展示

```
2 在使用Matplotlib时，需要安装第三方库
3 pip install matplotlib
4 基础案例：
5 """
6 Matplotlib:优秀的第三方框架
7 """
8 import matplotlib.pyplot as plt
9 """
10 figure:面板，plt中所有图像都位于figure对象中，一个图像只能有一个figure对象
11 """
12 # 创建面板
13 plt.figure(num='可视化',figsize=(5,3),dpi=100,facecolor='green',edgecolor='red')
14 # 创建数据
15 my_list = [2,3,4,5,6]
16 # 这组数据为纵坐标，横坐标自动生成
17 plt.plot(my_list)
18 # 面板展示
19 plt.show()
20 # 把生成图片保存本地
21 plt.savefig('test.png')
22 案例2:
23 import matplotlib.pyplot as plt
24 # 设置该图片为高清图
25 # 绘图图像,所有配置采用默认的
26 fig = plt.figure()
27 x=[1,2,3,4,5,6,7,8,9,10]
28 y=[1,18,9,4,3,5,7,10,12,3]
29 plt.plot(x,y)
30 # plt.show()
31 plt.savefig('test1.png')
32 案例3: 在Matplotlib中有两种画图方式:
33 import matplotlib.pyplot as plt
34
35 # 定义初始化数据
36 x=[1,2,3,4,5,6,7,8,9,10]
37 y=[1,18,9,4,3,5,7,10,12,3]
38
39 # 定义面板
40 fig1 = plt.figure()
41 fig2 = plt.figure()
42 # 采用matlab方式
43 # plt.plot(x,y)
```

```
44 # plt.show()
45 # 面向对象的方式
46 ax2 = plt.axes()
47 ax2.plot(x,y)
48 # plt.show()
49 plt.show()
50 案例4: 整合pandas做的图表
51 import matplotlib.pyplot as plt
52 import pandas as pd
53
54 # 读取 csv文件
55 df = pd.read_csv('start_time.csv',names=['start_time','count'])
56
57 # 创建面板
58 fig = plt.figure()
59 x=df['start_time'].values
60 y=df['count'].values
61 # 绘制图
62 plt.plot(x,y)
63 plt.show()
64 # plt.savefig('start_time.png')
65 案例4: 在plt中添加标记
66 'k'表示黑色;
67 'b'表示蓝色
68 'c' 表示青色
69 'g'表示绿色
70 'm'表示洋红
71 'r'表示红色
72 'y'黄色
73 'gray'或'grey'代表灰色
74 'lightgray'或'lightgrey'代表浅灰
75 代码:
76 from matplotlib import pyplot as plt
77
78 # 定义一个数据源
79 my_list=[2,7,5,6,9]
80 # 绘图
81 # plt.plot(my_list,'o:y') # 用参数来标记强调
82 plt.plot(my_list,marker='o')
83 plt.show()
84 案例5: 柱状图
85 from matplotlib import pyplot as plt
```

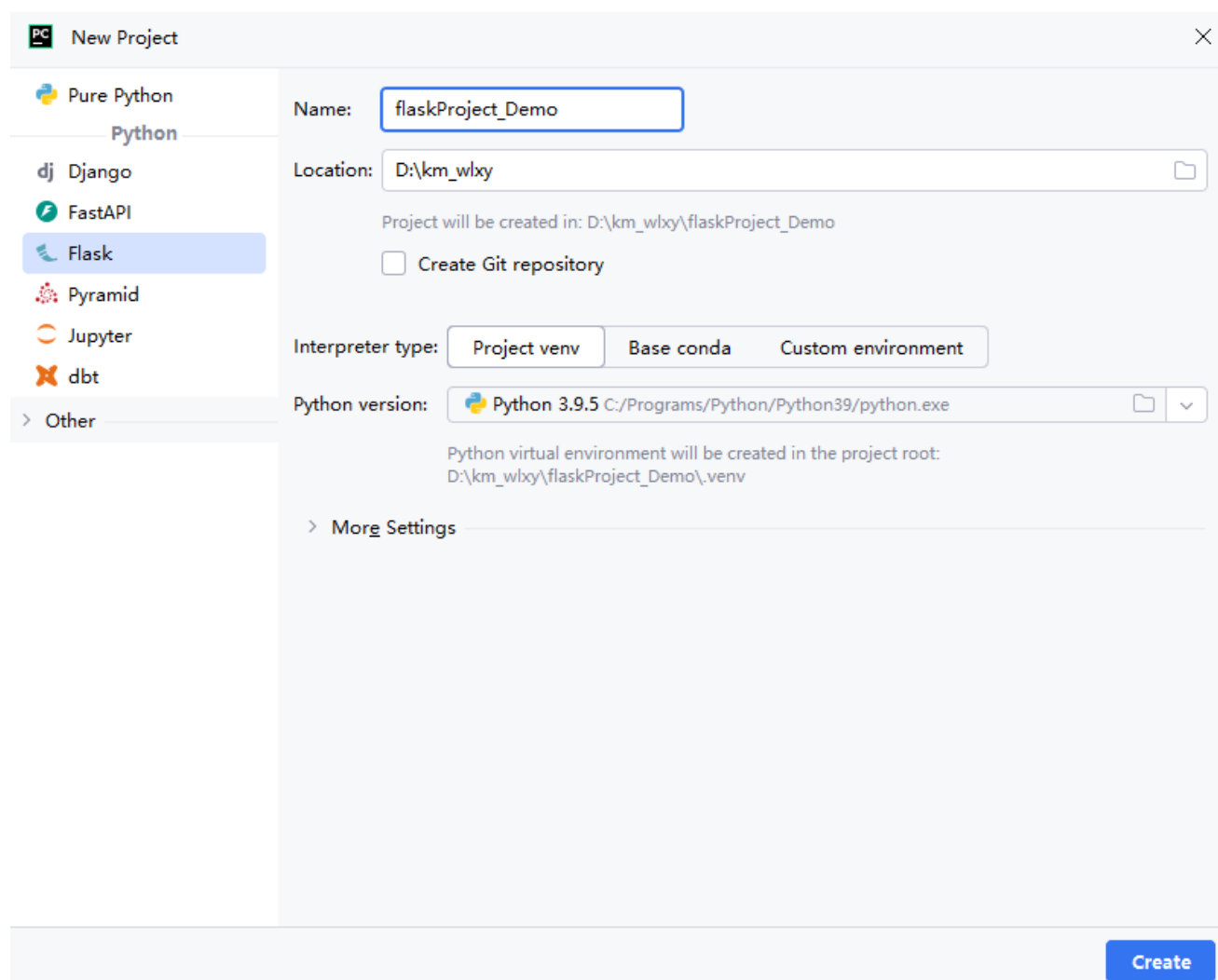
```
86 import pandas as pd
87 # 让图片显示中文
88 plt.rcParams['font.sans-serif'] = ['SimHei']
89 # 让图片可以显示负号
90 plt.rcParams['axes.unicode_minus'] = False
91
92 # 读取数据源
93 df = pd.read_csv('start_time.csv',names=['start_time','count'])
94
95 # 设置x轴的数据
96 x = df['start_time'].values
97 # 设置y轴数据
98 y = df['count'].values
99 # 创建柱状图
100 plt.title('统计分析上榜排行前五年份和上榜电影数')
101 plt.xlabel('年份/年')
102 plt.ylabel('数量/部')
103 plt.bar(x,y)
104 plt.show()
105 案例6: 饼图:
106 from matplotlib import pyplot as plt
107 import pandas as pd
108 # 让图片显示中文
109 plt.rcParams['font.sans-serif'] = ['SimHei']
110 # 让图片可以显示负号
111 plt.rcParams['axes.unicode_minus'] = False
112
113 # 读取数据源
114 df = pd.read_csv('start_time.csv',names=['start_time','count'])
115
116 # 设置名称
117 my_name = df['start_time'].values
118 # 设置值
119 my_value = df['count'].values
120 # 创建一个饼图
121 plt.pie(my_value,labels=my_name)
122 # 设置类别
123 plt.legend(loc='right')
124 plt.show()
125 作业1:
126 统计分析各个国家上榜电影数排行前十的国家和电影数量, 并且使用plt进行使用饼图来展示
127
```

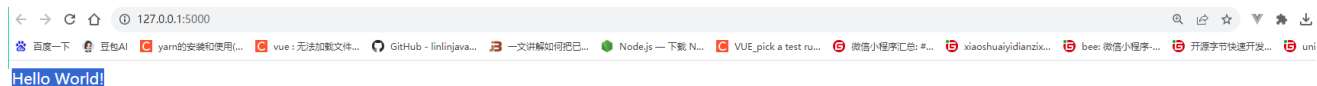
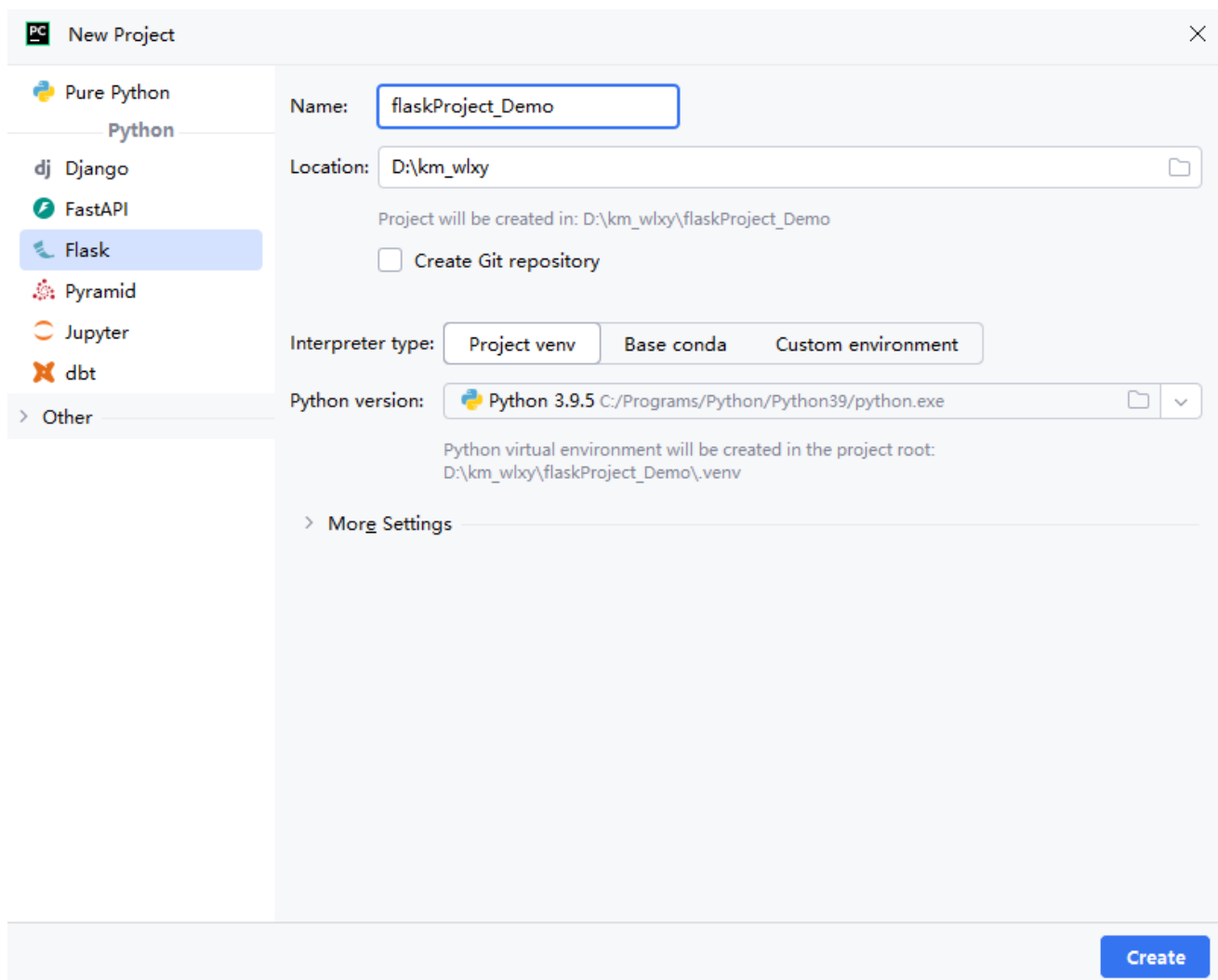
128 作业2：
129 统计电影最受欢迎的排行前十的电影名称以及评论数，使用折线图展示
130
131
132 作业3：
133 统计评分最高的十部电影，电影名称和评分，使用柱状图展示

3.9 Flask框架---pythonweb模块

- 1 Flask是python一个轻量级，常用于做服务，常用于前后端分离的工作。
- 2 Flask框架是如何应用的：

3.9.1 Flask框架的初体验





3.9.2 web框架概述

- 1 servlet+jsp----web框架
- 2 spring+springmvc-----目前整个互联网中应用最多框架
- 3 nodejs后端
- 4 通俗来讲，web框架主要作用连接前端和后端，数据传输
- 5 flask框架也是可以进行前后端的数据传输
- 6 目前主流开发模式：前后端分离
- 7 前端和后端，中间是可以通过json格式数据来进行传递
- 8 flask框架
 - 9 a、flask框架配置讲解
 - 10 b、flask框架路由，装饰器，映射

3.9.2 创建一个flask框架

```
1 a、创建一个普通的python项目
2 b、安装flask框架的库 : pip install flask
3 c、从零构建flask框架
4 """
5 导入相关库
6 """
7 from flask import Flask
8 # 创建一个flask对象
9 app = Flask(__name__)
10
11 # 路由, 装饰器, 主要映射访问地址和当前函数
12 @app.route('/my')
13 def my_test():
14     return "hello china"
15
16 if __name__ == '__main__':
17     # 启动flask框架
18     app.run()
```

3.9.3 flask框架中相关配置

```
1 第一块内容:
2 主要进行系统的相关配置:
3 # 创建一个flask对象,static_url_path 指的网页上访问路径,static_folder表示本地路径,确定本地项目的静态资源
4 app = Flask(__name__, static_url_path='/', static_folder='static')
5
6 第二块内容:
7 # 启动flask框架,debug=True 修改代码后不需要重新启动,host ip地址 port 端口号
8 app.run(debug=True, host='127.0.0.1', port=80)
9
10 第三块项目前后端分离: 涉及 跨域问题
11 步骤: 安装跨域库
```

```

12 (.venv) PS D:\km_wlxy\my_flask> pip install flask-cors
13 导入相关包:
14 # 导入跨域库
15 from flask_cors import CORS
16 # 解决跨域问题
17 CORS(app)
18
19 完整代码:
20 """
21 导入相关库
22 """
23 from flask import Flask
24 # 导入跨域库
25 from flask_cors import CORS
26
27 # 导入跨域的库
28
29 # 创建一个flask对象,static_url_path 指的网页上访问路径,static_folder表示本地路径,确定本地项目的静态资源
30 app = Flask(__name__, static_url_path='/', static_folder='static')
31 # 解决跨域问题
32 CORS(app)
33 # 路由,装饰器,主要映射访问地址和当前函数
34 @app.route('/my')
35 def my_test():
36     return "hello china"
37 @app.route('/test')
38 def test():
39     return "test"
40
41 if __name__ == '__main__':
42     # 启动flask框架,debug=True 修改代码后不需要重新启动,host ip地址 port 端口号
43     app.run(debug=True, host='127.0.0.1', port=80)

```

3.9.4 Flask框架中传值

```

1 get方式:
2 @app.route('/test2', methods=['GET', 'POST'])
3 def get_args():
4     # 获取get方式值

```

```

5     username = request.args.get('username')
6     password = request.args.get('password')
7     return '{}:{}'.format(username, password)
8 post方式:
9 login.html
10 <!DOCTYPE html>
11 <html lang="en">
12 <head>
13     <meta charset="UTF-8">
14     <title>登录页面</title>
15 </head>
16 <body>
17     <form action="login" method="post">
18         账号 : <input type="text" value="" name="username"><br>
19         密码 : <input type="password" value="" name="password"><br>
20         <input type="submit" value="登录">
21     </form>
22 </body>
23 </html>
24 后端:
25 @app.route('/login', methods=['GET', 'POST'])
26 def get_post():
27     # 通过post传值
28     username = request.form.get('username')
29     password = request.form.get('password')
30     return '{}:{}'.format(username, password)
31 注意: 编写代码时候, 函数名称不能相同

```

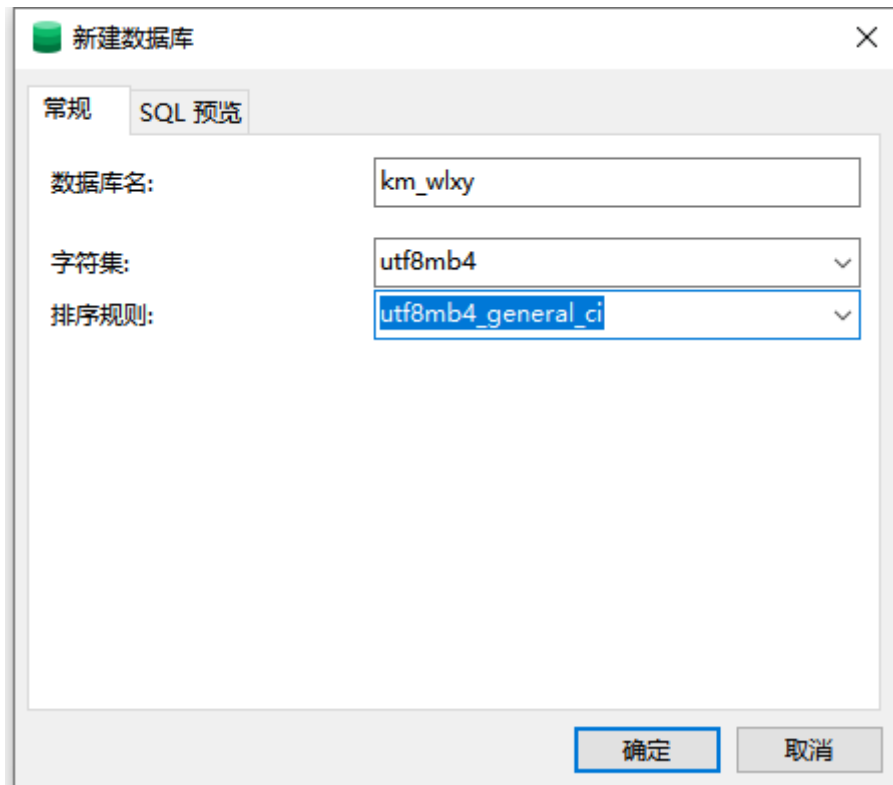
3.10 PyMysql讲解

```

1 什么关系型数据库?
2 在百度百科中, 所关系型数据库, 指的是以二维表格进行存储数据, 称之为关系型数据库。
3 name  age  sex
4 zs     18    男
5 ls     19    女
6 Sql指的是结构化查询语句
7 利用python语言来操作数据, python提供很多种库来操作数据库, 在这里推荐使用Pymysql来操作数据库
8 准备工作:

```

- 9 a、创建一个数据库
- 10 b、安装pymysql的包
- 11 `pip install pymysql`



3.10.1 利用sql语句完成表的创建

```
1 创建:
2 create table user_test(id int(4) primary key auto_increment,username
  varchar(255),password varchar(255))
3 案例:
4 """
5 创建数据库表
6 """
7 import pymysql
8
9 # 获取数据库连接
10 connect =
  pymysql.connect(host='127.0.0.1',port=3306,user='root',passwd='123456',db='km_wlxy')
11 # 创建游标
12 cursor =connect.cursor()
13 # 编写sql语句
14 sql = 'create table user_test(id int(4) primary key auto_increment,username
  varchar(255),password varchar(255))'
15 # 执行sql 语句
```

```
16 cursor.execute(sql)
17 # 提交执行
18 connect.commit()
19 # 关闭数据连接
20 connect.close()
21 print("创建数据库表成功")
```

3.10.2 利用pymysql在数据库中插入多条数据

```
1  """
2  在数据库表中插入数据
3  """
4  import pymysql
5
6  # 获取数据库连接
7  connect =
    pymysql.connect(host='127.0.0.1',port=3306,user='root',passwd='123456',db='km_wlxy')
8  # 创建游标
9  cursor =connect.cursor()
10 # 编写sql语句----%s表示一个占位符
11 sql = 'insert into user_test(username,password) values(%s,%s)'
12 my_args=('admin123','123456')
13 # 执行sql 语句
14 cursor.execute(sql,my_args)
15 # 提交执行
16 connect.commit()
17 # 关闭数据连接
18 connect.close()
19 print("创建数据库插入一条语句成功")
```

3.10.3 利用pymysql进行更新一条语句

```
1  """
2  在数据库表中更新数据
3  """
4  import pymysql
5
```

```
6 # 获取数据库连接
7 connect =
  pymysql.connect(host='127.0.0.1',port=3306,user='root',passwd='123456',db='km_wlxy')
8 # 创建游标
9 cursor =connect.cursor()
10 # 编写sql语句----%s表示一个占位符
11 sql = 'update user_test set username=%s,password=%s where id=%s'
12 my_args=('user1','123',2)
13 # 执行sql 语句
14 cursor.execute(sql,my_args)
15 # 提交执行
16 connect.commit()
17 # 关闭数据连接
18 connect.close()
19 print("创建数据库更新一条语句成功")
```

3.10.4 利用pymysql删除一条语句

```
1 """
2 在数据库表中更新数据
3 """
4 import pymysql
5
6 # 获取数据库连接
7 connect =
  pymysql.connect(host='127.0.0.1',port=3306,user='root',passwd='123456',db='km_wlxy')
8 # 创建游标
9 cursor =connect.cursor()
10 # 编写sql语句----%s表示一个占位符
11 sql = 'delete from user_test where id =%s'
12 my_args=(2)
13 # 执行sql 语句
14 cursor.execute(sql,my_args)
15 # 提交执行
16 connect.commit()
17 # 关闭数据连接
18 connect.close()
19 print("创建数据库删除一条语句成功")
```

3.10.5 利用pymysql进行数据的查询

```
1  """
2  在数据库表中更新数据
3  """
4  import pymysql
5
6  # 获取数据库连接
7  connect =
    pymysql.connect(host='127.0.0.1',port=3306,user='root',passwd='123456',db='km_wlxy')
8  # 创建游标
9  cursor =connect.cursor()
10 # 编写sql语句----%s表示一个占位符
11 sql = 'select * from user_test where id=%s'
12 my_args=(1)
13 # 执行sql 语句
14 cursor.execute(sql,my_args)
15 # 查询结果集
16 result = cursor.fetchone()
17 print(result,type(result),result[0])
18 # 关闭数据连接
19 connect.close()
20 print("创建数据库更新一条语句成功")
21 上述案例中：所有的返回结果是元组，返回结果为列表
22 """
23 在数据库表中更新数据
24 """
25 import pymysql
26
27 # 获取数据库连接
28 connect =
    pymysql.connect(host='127.0.0.1',port=3306,user='root',passwd='123456',db='km_wlxy')
29 # 创建游标----返回字典类型
30 cursor =connect.cursor(pymysql.cursors.DictCursor)
31 # 编写sql语句----%s表示一个占位符
32 sql = 'select * from user_test where id=%s'
33 my_args=[1]
34 # 执行sql 语句
35 cursor.execute(sql,my_args)
36 # 查询结果集
37 result = cursor.fetchone()
```

```

38 print(result,type(result),result['username'])
39 # 关闭数据连接
40 connect.close()
41 print("创建数据库更新一条语句成功")
42 上述结果返回是一个字典类型
43
44 查询所有：
45 """
46 在数据库表中更新数据
47 """
48 import pymysql
49
50 # 获取数据库连接
51 connect =
    pymysql.connect(host='127.0.0.1',port=3306,user='root',passwd='123456',db='km_wlxy')
52 # 创建游标----返回字典类型
53 cursor =connect.cursor(pymysql.cursors.DictCursor)
54 # 编写sql语句----%s表示一个占位符
55 sql = 'select * from user_test where id=%s'
56 my_args=[1]
57 # 执行sql 语句
58 cursor.execute(sql,my_args)
59 # 查询结果集
60 results = cursor.fetchall()
61 print(results)
62 for result in results:
63     print(result, type(result), result['username'])
64 # 关闭数据连接
65 connect.close()
66 print("创建数据库更新一条语句成功")

```

3.10.6 封装数据库的常用函数

```

1 封装：保护代码---封装来提高代码的复用性
2 resource.py:
3 # -----数据库相关配置信息-----
4 import os
5
6 DB_HOST='127.0.0.1'
7 DB_PORT=3306

```

```
8 DB_USER='root'
9 DB_PASSWORD='123456'
10 DB_NAME='km_wlxy'
11
12 # -----系统项目的路径-----
13 PRJ_PATH = os.path.realpath(__file__)
14
15
16
17 DB_Util.py
18 """
19 专门用于数据库的封装
20 """
21 import resource
22 import pymysql
23 """
24 获取数据库连接
25 """
26 def get_connect():
27     return pymysql.connect(host=resource.DB_HOST, port=resource.DB_PORT
28                             ,user=resource.DB_USER,password=resource.DB_PASSWORD,
29                             database=resource.DB_NAME)
30
31 """
32 关闭数据连接
33 """
34 def close_connect(connect):
35     connect.close()
36
37 # 封装 增删改
38 def cud_execute_sql(sql,my_args):
39     connect = get_connect()
40     cursor = connect.cursor()
41     cursor.execute(sql, my_args)
42     connect.commit()
43     close_connect(connect)
44     print("数据库发生改变....")
45 # 封装查询
46 def r_execute_sql(sql,my_args):
47     connect = get_connect()
48     # 查询要使用字典格式
49     cursor = connect.cursor(pymysql.cursors.DictCursor)
```

```
50     # 执行
51     cursor.execute(sql, my_args)
52     # 统一按照多条处理
53     reslust = cursor.fetchall()
54     # 关闭连接
55     close_connect(connect)
56     return reslust
57
58
59 测试:
60  """
61  导入相关库
62  """
63  from flask import Flask, request
64  # 导入跨域库
65  from flask_cors import CORS
66  from db import DB_Util
67  # 导入跨域的库
68
69  # 创建一个flask对象,static_url_path 指的网页上访问路径,static_folder表示本地路径,确定本地项目的静态资源
70  app = Flask(__name__, static_url_path='/', static_folder='static')
71  # 解决跨域问题
72  CORS(app)
73  # 路由,装饰器,主要映射访问地址和当前函数
74  @app.route('/my')
75  def my_test():
76      return "hello china"
77  @app.route('/test')
78  def test():
79      return "test"
80
81  @app.route('/test2', methods=['GET', 'POST'])
82  def get_args():
83      # 获取get方式值
84      username = request.args.get('username')
85      password = request.args.get('password')
86      return '{}:{}'.format(username, password)
87  @app.route('/login', methods=['GET', 'POST'])
88  def get_post():
89      # 通过post传值
90      username = request.form.get('username')
```

```

91     password = request.form.get('password')
92     return '{}:{}'.format(username, password)
93 @app.route('/list', methods=['GET', 'POST'])
94 def select_all():
95     sql = "select * from user_test"
96     res = DB_Util.r_execute_sql(sql, None)
97     return res
98 @app.route('/reg', methods=['GET', 'POST'])
99 def register():
100     username = request.args.get('username')
101     password = request.args.get('password')
102     sql="insert into user_test(username, password) values(%s,%s)"
103     my_args = (username, password)
104     DB_Util.cud_execute_sql(sql, my_args)
105     return 'ok'
106
107
108 if __name__ == '__main__':
109     # 启动flask框架,debug=True 修改代码后不需要重新启动,host ip地址 port 端口号
110     app.run(debug=True, host='127.0.0.1', port=80)

```

← → ↻ 🏠 ⓘ 127.0.0.1/reg?username=admin123456&password=123456

🐶 百度一下 🧑 豆包AI 📦 yarn的安装和使用(...) 📄 vue : 无法加载文件... 🌐 GitHub - linlinjava... 📖 一文讲解如何把已... 🟢 Node.js — 下



```
[
  {
    "id": 1,
    "password": "123456",
    "username": "admin"
  },
  {
    "id": 3,
    "password": "123456",
    "username": "admin123456"
  }
]
```

- 1 作业1:
- 2 完成更新测试
- 3 作业2:
- 4 完成删除测试
- 5 作业3:
- 6 完成根据用户名和密码查询测试

3.11 协同过滤

- 1 `pip freeze > requirements.txt` ---导出相对的库
- 2 `pip install -r requirements.txt`
- 3 协同过滤（**Collaborative Filtering**）是一种常用于推荐系统中的技术，它通过分析用户的历史行为或相似用户的行为来推荐新的物品。协同过滤算法主要分为基于用户的协同过滤（**User-Based Collaborative Filtering, UBCF**）和基于物品的协同过滤（**Item-Based Collaborative Filtering, IBCF**）。
- 4 在Python中，有多种库可以实现协同过滤算法，例如**scikit-learn**、**surprise**等。下面以**surprise**库为例，介绍如何使用协同过滤算法。
- 5 安装:
- 6 安装依赖库:
- 7 `(.venv) PS D:\km_wlxy\my_flask> pip install --upgrade numpy-i https://pypi.tuna.tsinghua.edu.cn/simple`

```
8 C:\Users\Administrator> pip install scikit-surprise
  -i https://pypi.tuna.tsinghua.edu.cn/simple
9 pip install numpy==1.25.0 scipy
10
11
12 案例:
13
```

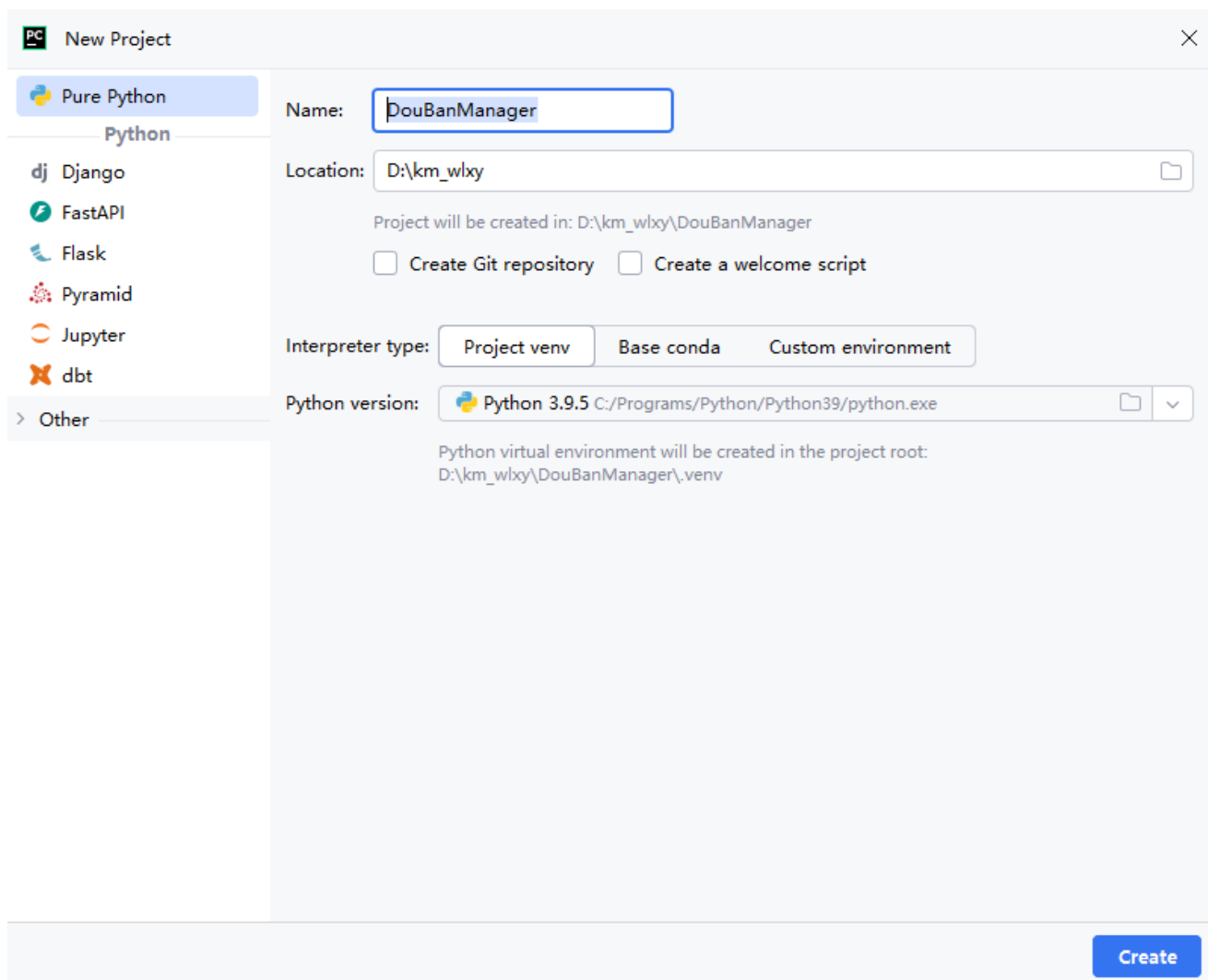
四、项目开发

4.1 需求分析

```
1 项目架构:
2 技术选型: 本项目采用python语言作为开发语言, 后端服务利用flask框架作为后端服务
3 项目数据库采用: mysql作为关系型数据库
4 前端: html+css+js----layui框架管理系统
5 本次项目主题, 根据豆瓣top250数据为原始数据来实现豆瓣电影管理系统----推荐系统
6 思路:
7     1、爬虫, 利用爬虫获取豆瓣官网top250的数据
8     2、pandas, 进行数据清洗和分析
9     3、matplotlib进行数据可视化, 生成图片
10    4、搭建flask框架, 进行前后端交互
11    5、利用pymysql操作数据库, 主要完成登录, 用户列表, 电影列表等功能模块
12    6、实现推荐功能(可选)
13 工具:
14 pip freeze > requirements.txt    ---导出相对的库
15 pip install -r requirements.txt
```

4.2 如何从零开始搭建项目

```
1 先创建一个Flask框架
```



安装相关库：

- 1 安装flask框架相关依赖库：
- 2 安装flask框架的核心库：
- 3 `(.venv) PS D:\km_wlxy\DouBanManager> pip install flask -i https://pypi.tuna.tsinghua.edu.cn/simple`
- 4 安装flask框架的跨域库：
- 5 `(.venv) PS D:\km_wlxy\DouBanManager> pip install flask-cors -i https://pypi.tuna.tsinghua.edu.cn/simple`
- 6 安装爬虫库：
- 7 `(.venv) PS D:\km_wlxy\DouBanManager> pip install requests -i https://pypi.tuna.tsinghua.edu.cn/simple`
- 8 `(.venv) PS D:\km_wlxy\DouBanManager> pip install beautifulsoup4 -i https://pypi.tuna.tsinghua.edu.cn/simple`
- 9 `(.venv) PS D:\km_wlxy\DouBanManager> pip install html5lib -i https://pypi.tuna.tsinghua.edu.cn/simple`
- 10 安装pandas库：
- 11 `(.venv) PS D:\km_wlxy\DouBanManager> pip install pandas -i https://pypi.tuna.tsinghua.edu.cn/simple`
- 12 安装数据可视化库：
- 13 `(.venv) PS D:\km_wlxy\DouBanManager> pip install matplotlib -i https://pypi.tuna.tsinghua.edu.cn/simple`

```
14 安装数据库库：
15 (.venv) PS D:\km_wlxy\DouBanManager> pip install pymysql -i
https://pypi.tuna.tsinghua.edu.cn/simple
16 如何导出项目中需要安装库：
17 (.venv) PS D:\km_wlxy\DouBanManager> pip freeze > requirements.txt
18 可以不用一个一个的进行安装了：
19 (.venv) PS D:\km_wlxy\DouBanManager> pip install -r .\requirements.txt
```

配置flask框架：

```
1 创建一个启动文件：
2 """
3 Flask框架的启动文件
4 """
5 from flask import Flask, request
6 from flask_cors import CORS
7
8 app = Flask(__name__)
9 # 解决跨域问题
10 CORS(app)
11
12 if __name__ == '__main__':
13     app.run(debug=True, host='127.0.0.1', port=8080)
```

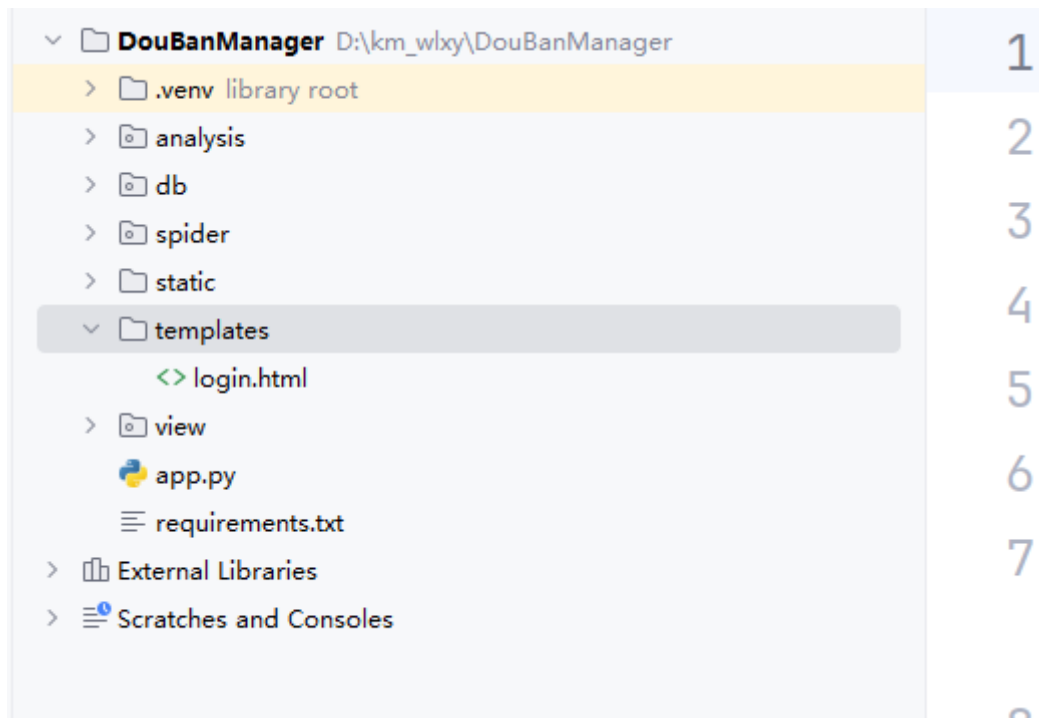
框架目录结构：

```
1 代码：
2 """
3 Flask框架的启动文件
4 """
5 from flask import Flask, request,render_template
6 from flask_cors import CORS
7
8 # 配置静态资源
9 app = Flask(__name__,static_url_path='/', static_folder='static')
10 # 解决跨域问题
11 CORS(app)
12
13 # 配置启动网页
```

```

14 @app.route('/')
15 def welcome():
16     return render_template("login.html")
17
18 if __name__ == '__main__':
19     app.run(debug=True, host='127.0.0.1', port=8080)

```



4.3 实现登录模块

创建一个数据库表：user表

对象user @km_wlxy (本地数据库) - 表

保存

添加字段

插入字段

删除字段

主键

上移

下移

字段

索引

外键

检查

触发器

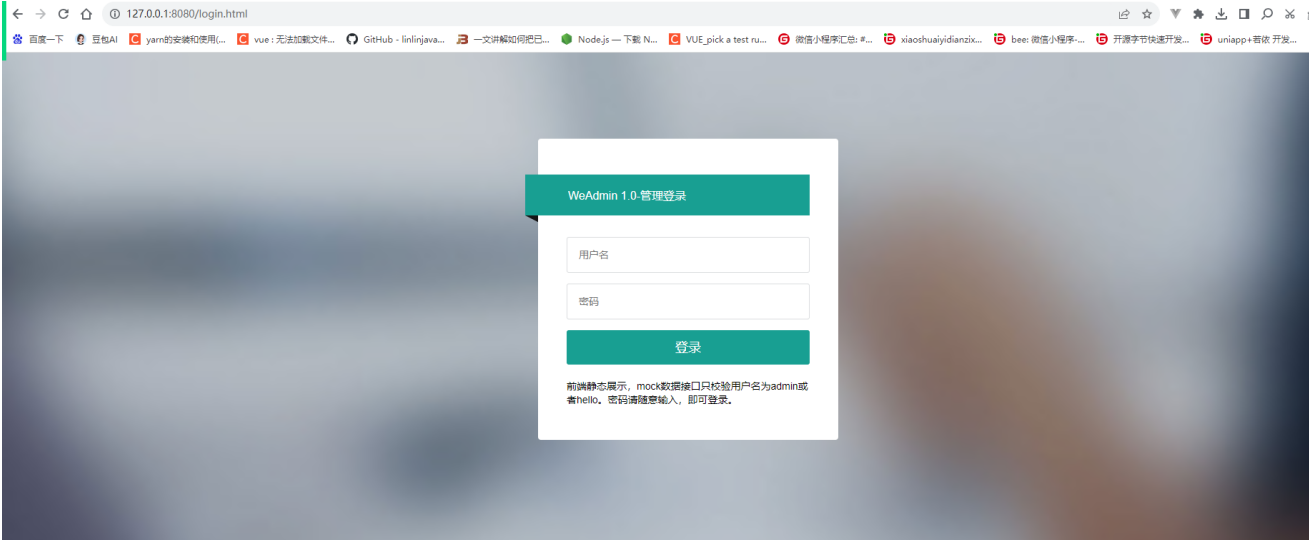
选项

注释

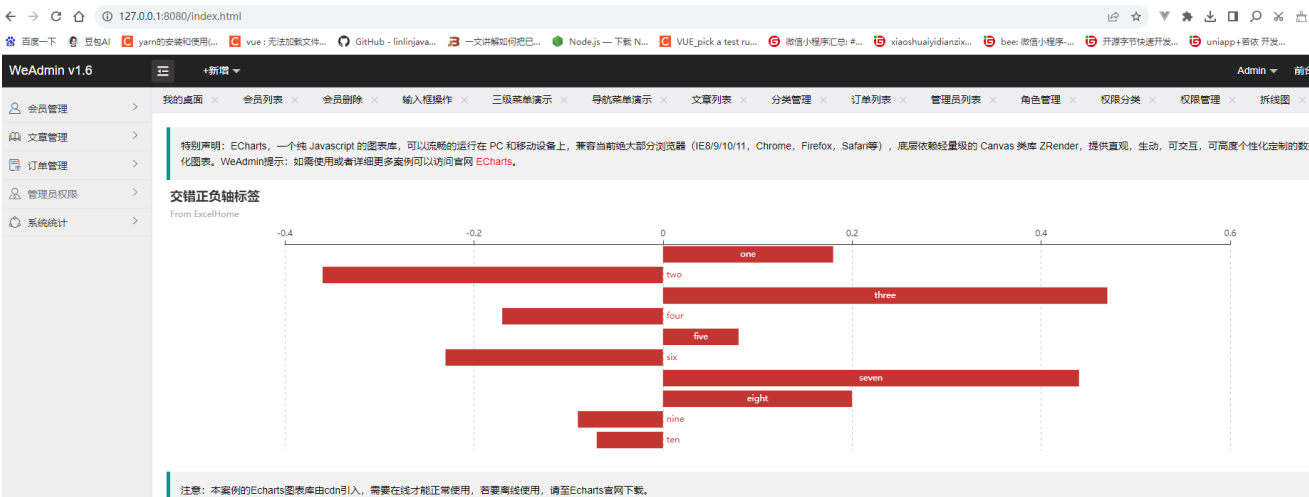
SQL 预览

名	类型	长度	小数点	不是 null	虚拟	键	注释
id	int			☒	☐	1	
username	varchar	255		☐	☐		
password	varchar	255		☐	☐		
phone	varchar	255		☐	☐		
email	varchar	255		☐	☐		
addr	varchar	255		☐	☐		
reg_time	varchar	255		☐	☐		
status	varchar	255		☐	☐		

登录界面：



登录成功后访问首页：



¹ 运行，启动项目，实现登录模块

防止页面出现开局刷新的情况，可以在使用修改admin.js文件

```

~ /
!(function () : void {
    console.log('打开或刷新后执行'); //sy-log
    if (!localStorage.getItem(key: 'username') && location.pathname.indexOf('/') === -1) {
        layer.msg('登录失效, 请重新登录~', function () : void {
            window.location.href = '/';
        });
    }
}

```

在admin.js文件的175行

4.4 Menu.json文件讲解

```

1  {
2      "status": 0,
3      "msg": "ok",
4      "data": [{
5          "id": 1,
6          "name": "会员管理",
7          "icon": "&#xe6b8;",
8          "url": "",
9          "children": [{
10             "id": 2,
11             "name": "会员列表",
12             "icon": "",
13             "url": "../pages/member/list.html"
14         }, {
15             "id": 3,
16             "name": "会员删除",
17             "icon": "",
18             "url": "../pages/member/del.html"
19         }, {
20             "id": 4,
21             "name": "会员管理",
22             "icon": "&#xe70b;",
23             "url": "",
24             "children": [{
25                 "id": 5,
26                 "name": "输入框操作",
27                 "icon": "",
28                 "url": "../pages/member/addInput.html"
29             }, {
30                 "id": 6,
31                 "name": "三级菜单演示",
32                 "icon": "",

```

```
33         "url": "./pages/404.html"
34     }, {
35         "id": 7,
36         "name": "导航菜单演示",
37         "icon": "&#xe70b;",
38         "url": "./pages/404.html"
39     }]
40 }
41 },
42 {
43     "id": 8,
44     "name": "文章管理",
45     "icon": "&#xe705;",
46     "url": "",
47     "children": [{
48         "id": 9,
49         "name": "文章列表",
50         "icon": "",
51         "url": "./pages/article/list.html"
52     }, {
53         "id": 10,
54         "name": "分类管理",
55         "icon": "",
56         "url": "./pages/article/category.html"
57     }]
58 },
59 {
60     "id": 11,
61     "name": "订单管理",
62     "icon": "&#xe723;",
63     "url": "",
64     "children": [{
65         "id": 12,
66         "name": "订单列表",
67         "icon": "",
68         "url": "./pages/order/list.html"
69     }]
70 },
71 {
72     "id": 13,
73     "name": "管理员权限",
74     "icon": "&#xe726;",
```

```
75     "url": "",
76     "children": [{
77         "id": 14,
78         "name": "管理员列表",
79         "icon": "",
80         "url": "../pages/admin/list.html"
81     },
82     {
83         "id": 15,
84         "name": "角色管理",
85         "icon": "",
86         "url": "../pages/admin/role.html"
87     },
88     {
89         "id": 16,
90         "name": "权限分类",
91         "icon": "",
92         "url": "../pages/admin/cate.html"
93     },
94     {
95         "id": 17,
96         "name": "权限管理",
97         "icon": "",
98         "url": "../pages/admin/rule.html"
99     }]
100 },
101 {
102     "id": 18,
103     "name": "系统统计",
104     "icon": "&#xe6ce;",
105     "url": "",
106     "children": [{
107         "id": 19,
108         "name": "拆线图",
109         "icon": "",
110         "url": "../pages/echarts/echarts1.html"
111     },
112     {
113         "id": 20,
114         "name": "柱状图",
115         "icon": "",
116         "url": "../pages/echarts/echarts2.html"
```

```
117     },
118     {
119         "id": 21,
120         "name": "地图",
121         "icon": "",
122         "url": "../pages/echarts/echarts3.html"
123     },
124     {
125         "id": 22,
126         "name": "饼图",
127         "icon": "",
128         "url": "../pages/echarts/echarts4.html"
129     },
130     {
131         "id": 23,
132         "name": "雷达图",
133         "icon": "",
134         "url": "../pages/echarts/echarts5.html"
135     },
136     {
137         "id": 24,
138         "name": "k线图",
139         "icon": "",
140         "url": "../pages/echarts/echarts6.html"
141     },
142     {
143         "id": 25,
144         "name": "热力图",
145         "icon": "",
146         "url": "../pages/echarts/echarts7.html"
147     },
148     {
149         "id": 26,
150         "name": "仪表图",
151         "icon": "",
152         "url": "../pages/echarts/echarts8.html"
153     },
154     {
155         "id": 27,
156         "name": "地图DIY实例",
157         "icon": "",
158         "url": "../pages/echarts/echarts9.html"
```

```
159         }}
160     }
161 ]
162 }
163 代表菜单栏和路径:
164 新的菜单栏:
165 {
166     "status": 0,
167     "msg": "ok",
168     "data": [{
169         "id": 1,
170         "name": "会员管理",
171         "icon": "&#xe6b8;",
172         "url": "",
173         "children": [{
174             "id": 2,
175             "name": "会员列表",
176             "icon": "",
177             "url": "./pages/member/list.html"
178         }]
179     },
180     {
181         "id": 13,
182         "name": "管理员权限",
183         "icon": "&#xe726;",
184         "url": "",
185         "children": [{
186             "id": 14,
187             "name": "管理员列表",
188             "icon": "",
189             "url": "./pages/admin/list.html"
190         }]
191     },
192     {
193         "id": 18,
194         "name": "系统统计",
195         "icon": "&#xe6ce;",
196         "url": "",
197         "children": [{
198             "id": 19,
199             "name": "拆线图",
200             "icon": "",
```

```
201         "url": "./pages/echarts/echarts1.html"
202     },
203     {
204         "id": 20,
205         "name": "柱状图",
206         "icon": "",
207         "url": "./pages/echarts/echarts2.html"
208     }]
209 }
210 ]
211 }
212
```

4.5 实现用户列表中列表展示和删除功能

1 列表展示:

2 思路:

- 3 a、查询数据库中所有用户的信息
- 4 b、转换成json格式数据
- 5 c、接受数据并展示

6 后端:

7

```
8 @app.route('/selectAllUser', methods=['GET', 'POST'])
```

```
9 def find_all_user():
```

```
10     sql = "select * from user"
```

```
11     # 调用工具类然后直接返回所有用户列表信息
```

```
12     result = DB_Util.r_execute_sql(sql, None)
```

```
13     response_json = ''
```

```
14     if len(result)>0:# 表示列表中不为空
```

```
15         response_json=Str_To.Json(1,result,'success').to_json()
```

```
16     else:
```

```
17         response_json=Str_To.Json(0,{},'用户信息为空, 请前往数据库中添加').to_json()
```

```
18     return response_json
```

19 前端:

```

"url": "",
"children": [{
    "id": 2,
    "name": "会员列表",
    "icon": "",
    "url": "./pages/member/list2.html"
}]

```

修改会员列表路径

4.6 电影数据初始化

```

1  """
2  该python文件主要用于初始化电影数据表
3  [title, img_url, dy, start_time, addr, type, grade, rating_count, rating_mes]
4  """
5  import csv
6  import os
7
8  import DB_Util
9  from resource import PRJ_PATH
10
11  create_sql = ("create table if not exists movie(id int(4) primary key
12              auto_increment,title varchar(255),"
13              "img_url varchar(255),dy varchar(255),start_time varchar(255),addr
14              varchar(255),"
15              "type varchar(255),grade double,rating_count int(4),rating_mes
16              varchar(255))");
17  # 创建数据库表
18  DB_Util.cud_execute_sql(create_sql,None)
19  # 文件夹路径
20  dir = os.path.join(PRJ_PATH,"douban")
21  # 初始化数据库表
22  def init():
23
24      with open('{}/douban.csv'.format(dir),'r',encoding='utf-8') as f:
25          reader = csv.reader(f)
26          for row in reader:
27              sql = ("insert into movie(title, img_url, dy, start_time, addr, type, grade,
28                  rating_count, rating_mes) "

```

```
25         "values(%s,%s,%s,%s,%s,%s,%s,%s,%s,%s) ")
26         DB_Util.cud_execute_sql(sql,row)
27         print("插入一条语句")
28
29 if __name__ == '__main__':
30     init()
```